
ADVANTEST®

ADVANTEST CORPORATION

ASX/U-51
Communication Control
Library Manual

MANUAL NUMBER 8431496-02

Applicable Systems

T5593

T5592

T5586

T5585

T5377

T5376

T5375

Trademarks and Registered Trademarks

- ADVANTEST, ADVANsite, ATL and ATLworks are trademarks of Advantest Corporation.
- Sun, Sun Microsystems, the Sun logo, Ultra, Solaris and NFS are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries.
- All SPARC trademarks are used under license of SPARC International, Inc. and are also trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries. Products bearing SPARC trademarks are based on an architecture developed by Sun Microsystems, Inc.
- IBM, PowerPC and PowerPC Architecture are trademarks or registered trademarks of International Business Machines Corp.
- UNIX is a registered trademark in the United States and other countries, licensed exclusively through The Open Group.
- VxWorks, Wind River Systems, the Wind River Systems logo, wind, CrossWind, Tornado and WindSh are trademarks or registered trademarks of Wind River Systems, Inc.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Revision History

Rev.	Date	Notes
01	May 15/03	
02	Jul 25/03	

Table of Contents

Preface	Preface-1
Regarding This Manual	Preface-1
Related Manual	Preface-1
Typographic Convention	Convention-1
Conventions for Command-Line and Program Examples	Convention-1
Indicators for Operating the GUI	Convention-2
Indicators for Keyboard Operation	Convention-2
Reference Indicators	Convention-3
1. Overview	1-1
1. 1 Introduction	1-1
1. 2 System Overview	1-2
1. 3 Operating Environment	1-4
1. 3. 1 Required Hardware	1-4
1. 3. 2 Required Software	1-4
1. 3. 3 Applicable Versions	1-4
2. Library Usage Method	2-1
2. 1 Environment Setup	2-1
2. 1. 1 Setting Environment Variables	2-1
2. 1. 2 GPIB Environment Settings	2-1
2. 2 Programming Method	2-2
2. 3 Compile Method	2-3
3. Tester Control Library	3-1
3. 1 Overview	3-1
3. 1. 1 HANDLER/PROBER Information Setting Functions	3-1
3. 1. 2 Initialization Functions	3-1

3. 1. 3	Program Control Functions	3-2
3. 1. 4	Measurement Result Processing Functions	3-2
3. 1. 5	Other Functions	3-4
3. 2	HANDLER/PROBER Information Setting Functions	3-5
3. 2. 1	AT_set_handler_request()	3-5
3. 2. 2	AT_set_prober_request()	3-7
3. 2. 3	AT_set_prober_address()	3-9
3. 2. 4	AT_set_prober_location()	3-10
3. 3	Initial Setting Library	3-11
3. 3. 1	AT_set_praname()	3-11
3. 3. 2	AT_set_test_mode()	3-13
3. 3. 3	AT_set_fk()	3-15
3. 3. 4	AT_set_datalog_mode()	3-16
3. 3. 5	AT_set_test_number()	3-17
3. 3. 6	AT_set_init()	3-18
3. 3. 7	AT_set_test_dut()	3-19
3. 3. 8	AT_set_DEVICE()	3-20
3. 3. 9	AT_set_LOT()	3-21
3. 3.10	AT_set_SUBLOT()	3-22
3. 3.11	AT_set_DUTIF()	3-23
3. 3.12	AT_set_OPERAT()	3-24
3. 3.13	AT_set_COMMEN()	3-25
3. 3.14	AT_set_WAFER()	3-26
3. 3.15	AT_read_DEVICE()	3-27
3. 3.16	AT_read_LOT()	3-28
3. 3.17	AT_read_SUBLOT()	3-29
3. 3.18	AT_read_DUTIF()	3-30
3. 3.19	AT_read_OPERAT()	3-31
3. 3.20	AT_read_COMMEN()	3-32
3. 3.21	AT_read_WAFER()	3-33
3. 3.22	AT_set_opdat()	3-34
3. 3.23	AT_read_opdat()	3-35
3. 3.24	AT_clear_program()	3-36
3. 3.25	AT_read_pbid()	3-37
3. 3.26	AT_read_pbtype()	3-38
3. 4	Program Control Library	3-39
3. 4. 1	AT_start_program()	3-39
3. 4. 2	AT_reset_program()	3-40
3. 4. 3	AT_wait_program()	3-41
3. 4. 4	AT_read_test_status()	3-42
3. 4. 5	AT_start_subprogram()	3-43
3. 5	Measurement Results Processing Library	3-44

3. 5. 1	AT_read_fail_dut()	3-44
3. 5. 2	AT_read_category_data()	3-46
3. 5. 3	AT_read_sort_data()	3-48
3. 5. 4	AT_read_total_counter()	3-49
3. 5. 5	AT_read_fail_counter()	3-50
3. 5. 6	AT_read_category_counter()	3-51
3. 5. 7	AT_read_sort_counter()	3-52
3. 5. 8	AT_read_test_fail_counter()	3-53
3. 5. 9	AT_read_sample_number()	3-54
3. 5.10	AT_read_wafer_total_counter()	3-55
3. 5.11	AT_read_wafer_fail_counter()	3-56
3. 5.12	AT_read_wafer_category_counter()	3-57
3. 5.13	AT_read_wafer_counter()	3-58
3. 5.14	AT_read_slice_number()	3-59
3. 5.15	AT_set_total_counter()	3-60
3. 5.16	AT_set_fail_counter()	3-61
3. 5.17	AT_set_category_counter()	3-62
3. 5.18	AT_set_sort_counter()	3-63
3. 5.19	AT_set_test_fail_counter()	3-64
3. 5.20	AT_set_sample_number()	3-65
3. 5.21	AT_set_wafer_total_counter()	3-66
3. 5.22	AT_set_wafer_fail_counter()	3-67
3. 5.23	AT_set_wafer_category_counter()	3-68
3. 5.24	AT_set_wafer_counter()	3-69
3. 5.25	AT_set_slice_number()	3-70
3. 5.26	AT_clear_total_counter()	3-71
3. 5.27	AT_clear_fail_counter()	3-72
3. 5.28	AT_clear_category_counter()	3-73
3. 5.29	AT_clear_sort_counter()	3-74
3. 5.30	AT_clear_test_fail_counter()	3-75
3. 5.31	AT_clear_sample_number()	3-76
3. 5.32	AT_clear_wafer_total_counter()	3-77
3. 5.33	AT_clear_wafer_fail_counter()	3-78
3. 5.34	AT_clear_wafer_category_counter()	3-79
3. 5.35	AT_clear_wafer_counter()	3-80
3. 5.36	AT_clear_slice_number()	3-81
3. 6	Other Functions	3-82
3. 6. 1	AT_set_lock()	3-82
3. 6. 2	AT_reset_lock()	3-84
3. 7	Tester Utilities	3-85
3. 7. 1	WROPDAT	3-85
3. 7. 2	RDOPDAT	3-86

4. GPIB Control Library	4-1
4. 1 Overview	4-1
4. 1. 1 Functions for Controlling GPIB	4-1
4. 1. 2 Functions for Controlling the PROBER	4-1
4. 1. 3 Functions for Controlling the HANDLER	4-2
4. 2 Functions for Controlling GPIB	4-3
4. 2. 1 AT_set_GPIB_address()	4-3
4. 2. 2 AT_set_GPIB_timeout()	4-4
4. 2. 3 AT_send_GPIB_data()	4-6
4. 2. 4 AT_read_GPIB_data()	4-7
4. 2. 5 AT_read_GPIB_status_byte()	4-8
4. 3 Functions for Controlling the PROBER	4-9
4. 3. 1 AT_select_prober()	4-9
4. 3. 2 AT_read_prober_machineID()	4-10
4. 3. 3 AT_read_prober_lotname()	4-11
4. 3. 4 AT_read_prober_waferID()	4-12
4. 3. 5 AT_read_prober_location()	4-13
4. 3. 6 AT_read_prober_address()	4-14
4. 3. 7 AT_read_prober_onwafer()	4-15
4. 3. 8 AT_set_prober_Zup()	4-17
4. 3. 9 AT_set_prober_Zdown()	4-18
4. 3.10 AT_start_prober_test()	4-19
4. 3.11 AT_send_prober_test_result()	4-20
4. 3.12 AT_set_prober_next_die()	4-22
4. 3.13 AT_set_prober_target_die()	4-23
4. 3.14 AT_set_prober_wafer_unload()	4-24
4. 4 Functions for Controlling the HANDLER	4-25
4. 4. 1 AT_read_handler_status()	4-25
4. 4. 2 AT_set_handler_SRQMASK()	4-26
4. 4. 3 AT_read_handler_SRQKIND()	4-27
4. 4. 4 AT_set_handler_SRQ_answer()	4-28
4. 4. 5 AT_read_handler_devicename()	4-29
4. 4. 6 AT_read_handler_lotname()	4-30
4. 4. 7 AT_set_handler_DUTNUMSEL()	4-31
4. 4. 8 AT_read_handler_DUTNUMSEL()	4-32
4. 4. 9 AT_read_handler_temp()	4-33
4. 4.10 AT_set_handler_PBTYP()	4-34
4. 4.11 AT_set_handler_TESTCNT()	4-35
4. 4.12 AT_read_handler_retest_category()	4-36
4. 4.13 AT_read_handler_DOUBLETEST_category()	4-37
4. 4.14 AT_set_handler_TESTALARM()	4-38

4. 4.15	AT_read_handler_testdut()	4-39
4. 4.16	AT_set_handler_sort()	4-40
4. 4.17	AT_clear_handler_retest_sort()	4-41

5. Error Code Quick Reference	5-1
--	------------

Appendix 1.GPIB Environment Settings	A1-1
---	-------------

A1. 1	Starting the Configuration Utility	A1-1
A1. 2	Changing the GPIB Device Name	A1-3
A1. 3	Setting GPIB Device Properties	A1-7
A1. 4	Ending the Configuration Utility	A1-8

List of Figures	F-1
------------------------------	------------

List of Tables	T-1
-----------------------------	------------

Index	I-1
--------------------	------------

Preface

Regarding This Manual

This manual describes details of library functions provided by the communication control library.

Related Manual

ASX/U-51 Hybrid Controller User's Manual(No. : 8409598)

Typographic Convention

Conventions for Command-Line and Program Examples

- Command-line and program examples are printed in a different font than the main body text so that they can be easily distinguished. Similarly, different styles are also used for user input and screen output.
- The command prompt for someone logged in as a general user is %; it is # for a superuser.
- Command execution is indicated by the return symbol (↵).
- Options which may be omitted are indicated by [].
- ... indicates that the item immediately before it can be specified repeatedly; [] indicates that omission is allowed.
- ... in a program example indicates that a source code description has been omitted.

Table C-1 Conventions for Command-Line and Program Examples

Typeface or Symbol	Description	Example
<code>abcd123</code>	Command, directory, and file names, as well as screen output and program examples, are shown.	To check the existence of the <code>.login</code> file, execute the <code>ls -a</code> command on the command line.
<code>abcd123</code>	Shows the character string entered by the user.	system% <code>su</code> ↵ password:
<i><code>abc123</code></i>	Shows a character string which the user must determine at the time of input.	system% <i><code>rm filename</code></i> ↵
	is the separator symbol for OR. When multiple expressions are separated by , it means to select only one of them.	<code>stnNum(int stn_num NONE)</code>
...	When this symbol appears, it indicates that the item immediately before it can be specified repeatedly. When it appears in a program example, it indicates that a source code description has been omitted.	<code>option(LCDPINOPTION, ...)</code>
[]	When this symbol appears, the specification enclosed in [] can be omitted.	<code>Get_Tof1(int dfn_adr, ARRAY<int> ary_data[,int adr_cnt])</code>

Typeface or Symbol	Description	Example
%	When the login account belongs to a general user, a C-shell command prompt is displayed.	system% ls ↵
#	When the login account belongs to a superuser, a Bourne-shell command prompt is displayed.	# ls ↵
↵	Shows the [Return] having been pressed on a command line.	system% ls ↵
[]	Options which may be omitted are shown by [] in the command line.	system% ls [-a] ↵

Indicators for Operating the GUI

The name of the option selection area of Graphical User Interface (GUI) screen is enclosed in the [] symbol. Also, operations for selecting commands within menus are indicated by an arrow symbol (→).

Table C-2 Indicators for Operating the GUI

Typeface or Symbol	Description	Example
[]	Shows the name of the option selection area on the GUI screen.	[File] menu [OK] button
[]→[]	Shows commands in a menu	[File]→[Exit]

Indicators for Keyboard Operation

Keyboard keys are enclosed in the [] symbol. When multiple keys are simultaneously depressed, the plus sign (+) is used as an indicator; multiple keys pressed in sequence are indicated by a comma.

Table C-3 Indicators for Keyboard Operation

Typeface or Symbol	Description	Example
[]	Shows keyboard keys	[Return] key
[Ctrl]	Shows the [Control] key.	[Ctrl] key
+	Shows a second key pressed while a first one is held down when keys are connected by the plus (+) sign.	[Ctrl]+[d]

Typeface or Symbol	Description	Example
,	Shows each key sequentially when keys are delimited by commas.	[Ctrl]+[x], [y]

Reference Indicators

Reference Indicators use double-quote (") symbols.

Table C-4 Reference Indicators

Typeface or Symbol	Description	Example
" "	Indicates chapter titles and section titles of a manual to refer to.	For details, refer to 2, "Restarting and Quitting".
<i>Italic</i>	Indicates the title of a manual to refer to.	For details, refer to <i>Installation guide</i> .

1. Overview

This chapter describes the system configuration and operating environment in using the communication control library.

1. 1 Introduction

The communication control library was developed to control handlers, probers, and other measuring instruments through GPIB asynchronously with the tester.

Before the communication control library is developed, the tester controller has been used to control handlers and probers. Therefore, the functions, which cannot be controlled by the tester controller, are controlled from test programs. However, the available execution environment of these test programs is restricted. The communication control library is capable of creating unrestricted control programs.

When two test stations connected to a probe are used for simultaneous measurements and a dummy start occurs in one of stations due to a wafer end, the other station suspends measuring. The communication control library allows the tester to perform dummy start processes and device measurements simultaneously.

The communication control library was developed assuming that the tester is used for device measurements and the workstation EWS is used to control peripheral equipment.

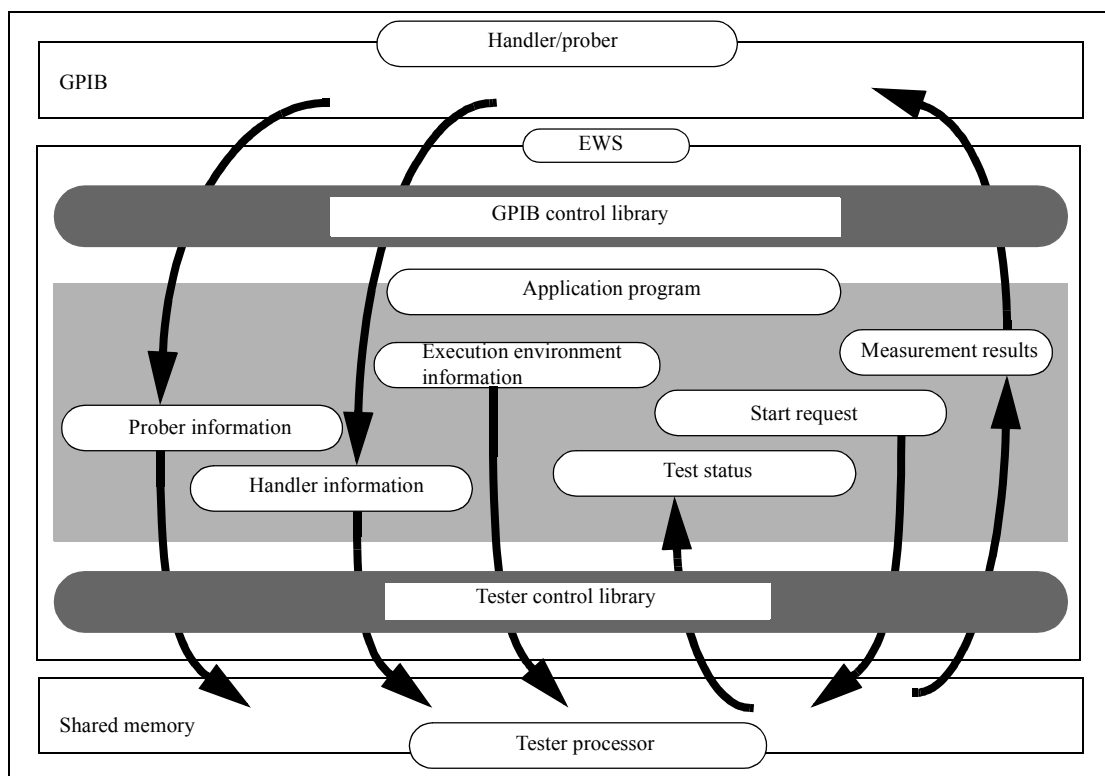
1. 2 System Overview

The communication control library consists of a GRIP control library and a tester control library. The GPIB control library is used to control handlers and probers through a GPIB interface. This library allows acquisition of start requests, setting of fail data, and other functions.

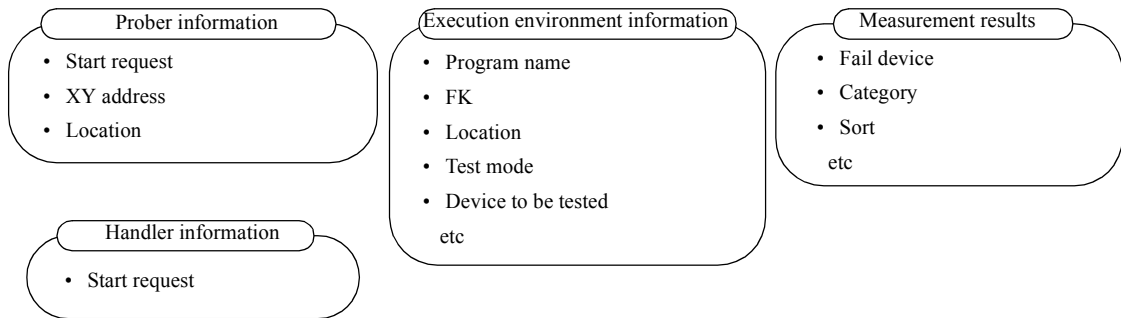
The tester control library is used to control test programs through the memory of the test processor (shared memory). Initial settings and program start of course allow the acquisition of measurement results and data processing.

The language used is the C language, and the handler, prober, and test program are controlled from the EWS.

Figure 1-1 System Block Diagram



The information items have the following structure.



1. 3 Operating Environment

1. 3. 1 Required Hardware

- Test system
An ADVANsite system (T5593, T5592, T5586, T5585, T5376, or T5375) loaded with TP5 or TP6 as the tester processor is required.
- EWS
Either of the following models is required.
 - Ultra-5
 - Blade-100
 - Blade-150
- GPIB Interface
The GPIB interface (National Instruments made) and GPIB driver software (NI-488.2 Version 2.2) (H3-6848) to be inserted into the PCI slot of the EWS are required.
They are not necessary when only the tester control library is used.

1. 3. 2 Required Software

- C compiler
Forte C 6 update 2 is required.

1. 3. 3 Applicable Versions

Execution is possible using ASX/U-51 REV 6.02 and later.

2. *Library Usage Method*

This chapter describes environment settings for using the library and methods to create programs using the library.

2. 1 Environment Setup

Before using the communication control library, several setup procedures are required.

! Important

The ASX/U-51 operation system must have been installed to use the communication control library, and the ASX/U-51 operation system, NI GPIB driver, and C compiler must have been installed to use the GPIB control library.

→ For more information on the installing method, refer to the appropriate installation manual.

2. 1. 1 Setting Environment Variables

The search path must be set to dynamically link the communication control library to the application program.

Add AT_ASYSTEM in the LD_LIBRARY_PATH environment variable.

```
% setenv LD_LIBRARY_PATH ${AT_ASYSTEM}:${LD_LIBRARY_PATH}
```

2. 1. 2 GPIB Environment Settings

In the GPIB control library, in order to access GPIB I/F, a specific name (logical device name) is used. Perform set up in accordance with [Appendix 1. "GPIB Environment Settings" on page A1-1](#).

2. 2 Programming Method

For using the communication control library it is required to include the header file "testerlib.h" specific for the tester control library and the header file "gpiblib.h" specific for the GPIB control library. Header files have been installed in \$AT_ASYSYSTEM/AT.

When only the tester control library is used

```
#include "AT/testerlib.h"
:
main() {
:
}
```

When only the GPIB control library is used

```
#include "AT/gpiblib.h"
:
main() {
:
}
```

When both tester control library and GPIB control library are used

```
#include "AT/testerlib.h"
#include "AT/gpiblib.h"
:
main() {
:
}
```

2. 3 Compile Method

For using the communication control library, it is required to link the tester control library "libat-tester.so" and the GPIB control library "libatgplib.so" to the application program.

When only the tester control library is used

```
% cc abc.c -o abc -I$AT_ASYSYSTEM -lattester
```

When only the GPIB control library is used

```
% cc abc.c -o abc -I$AT_ASYSYSTEM -latgplib
```

When both tester control library and GPIB control library are used

```
% cc abc.c -o abc -I$AT_ASYSYSTEM -lattester -latgplib
```

! Important

The tester controller must be launched first to use the tester control library.

Launch the tester controller first from a window other than the window for executing the application program, and then execute the application.

3. *Tester Control Library*

The tester control library consists of functions that control the test system from an EWS, enabling program execution, setting of an operation mode, reading of measurement results, and other processing.

3. 1 Overview

The tester control library is roughly divided into the following five function groups. This section describes overviews of individual function groups.

- HANDLER/PROBER information setting functions
- Initialization functions
- Program control functions
- Measurement result processing functions
- Other functions

3. 1. 1 HANDLER/PROBER Information Setting Functions

This group of functions sets the information concerning HANDLERs and PROBERs such as start requests and location data to the shared memory.

Table 3-1 List of HANDLER/PROBER Information Setting Functions

Function name	Function
AT_set_handler_request()	Sets the handler start request
AT_set_prober_request()	Sets the prober start request
AT_set_prober_address()	Sets the X/Y addresses of the prober.
AT_set_prober_location()	Sets the prober location

3. 1. 2 Initialization Functions

This group of functions performs initial setting of the execution program name and function keys and other initialization processing for program execution.

Table 3-2 List of Initialization Functions

Function name	Function
AT_set_proname()	Sets the execution program name

Function name	Function
AT_set_test_mode()	Sets the test mode
AT_set_fk()	Sets the function keys
AT_set_datalog_mode()	Sets the data log mode
AT_set_test_number()	Sets the test number
AT_set_init()	Turns on the INIT/CAL INIT lamp of CBOX
AT_set_test_dut()	Sets the test target DUT
AT_set_opdat()	Sets the operator data
AT_read_opdat()	Reads the operator data
AT_clear_program()	Clears the program area
AT_read_pbid()	Reads the PBID
AT_read_pbtype()	Reads the PBTYPE

3. 1. 3 Program Control Functions

This group of functions controls program execution such as start or forced suspension of a test program.

Table 3-3 List of Program Control Functions

Function name	Function
AT_start_program()	Executes the program
AT_reset_program()	Forcibly suspends the program
AT_wait_program()	Causes the system to wait until the program execution ends
AT_read_test_status()	Reads the test status
AT_start_subprogram()	Executes the subroutine program

3. 1. 4 Measurement Result Processing Functions

This group of functions reads out the measurement results including failed device information and category data.

Table 3-4 List of Measurement Result Processing Functions

Function name	Function
AT_read_fail_dut()	Reads failed device information
AT_read_category_data()	Reads the category data

Function name	Function
AT_read_sort_data()	Reads the sort data
AT_read_total_counter()	Reads the total counter data
AT_read_fail_counter()	Reads the fail counter data
AT_read_category_counter()	Read the category counter data
AT_read_sort_counter()	Reads the sort counter data
AT_read_test_fail_counter()	Reads the test fail counter data
AT_read_sample_number()	Reads the sample number
AT_read_wafer_total_counter()	Reads the total counter data for a wafer
AT_read_wafer_fail_counter()	Reads the fail counter data for a wafer
AT_read_wafer_category_counter()	Reads the category counter data for a wafer
AT_read_wafer_counter()	Reads the number of wafers
AT_read_slice_number()	Reads the slice number
AT_set_total_counter()	Sets the total counter
AT_set_fail_counter()	Sets the fail counter
AT_set_category_counter()	Sets the category counter
AT_set_sort_counter()	Sets the sort counter
AT_set_test_fail_counter()	Sets the test fail counter
AT_set_sample_number()	Sets the sample number
AT_set_wafer_total_counter()	Sets the total counter for a wafer
AT_set_wafer_fail_counter()	Setts the fail counter for a wafer
AT_set_wafer_category_counter()	Sets the category counter for a wafer
AT_set_wafer_counter()	Sets the number of wafers
AT_set_slice_number()	Sets the slice number
AT_clear_total_counter()	Clears the total counter
AT_clear_fail_counter()	Clears the fail counter
AT_clear_category_counter()	Clears the category counter
AT_clear_sort_counter()	Clears the sort counter
AT_clear_test_fail_counter()	Clears the test fail counter
AT_clear_sample_number()	Clears the sample number
AT_clear_wafer_total_counter()	Clears the total counter for a wafer
AT_clear_wafer_fail_counter()	Clears the fail counter for a wafer

Function name	Function
AT_clear_wafer_category_counter()	Clears the category counter for a wafer
AT_clear_wafer_counter()	Clears the number of wafers
AT_clear_slice_number()	Clears the slice number

3. 1. 5 Other Functions

This group of functions is not directly related to device measurement.

Table 3-5 List of Other Functions

Function name	Function
AT_set_lock()	Locks the application program
AT_reset_lock()	Unlocks the application program

3. 2 HANDLER/PROBER Information Setting Functions

3. 2. 1 AT_set_handler_request()

Sets the start request from the handler to the shared memory.

```
int AT_set_handler_request(int stno, unsigned int status);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int status

Specifies the start request data.

Specify the start request data by using the following mnemonics. Multiple statuses can be set simultaneously by delimiting them with a vertical bar (|).

Mnemonic	Request content
HL_NO_REQUEST	No request
HL_NORMAL_START	Normal start
HL_MANUAL_START	Manual start
HL_LOT_START	Lot start
HL_LOT_END	Lot end
HL_FINAL_LOT_END	Final lot end
HL_DOUBLE_TEST	Double test
HL_RETEST_LOT_START	Lot start (retest)
HL_RETEST1_START	Retest (1st)
HL_RETEST2_START	Retest (2nd)

Description

This function sets the start request sent from the handler indicated by *status* to the station specified by *stno*.

The tester controller uses this information to control the test program.

The most recently set information is retained until the information is reset by using the `AT_set_handler_request` function (refer to [3. 2. 1 "AT_set_handler_request\(\)" on page 3-5](#)) or `AT_set_prober_request` function (refer to [3. 2. 2 "AT_set_prober_request\(\)" on page 3-7](#)). In addition, the information set by the `AT_set_test_dut` function (refer to [3. 3. 7 "AT_set_test_dut\(\)" on page 3-19](#)) is cleared.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

This information can be read out with a `READ HANDLER` statement.

This statement is valid only when the `HANDLER` is described in the socket program.

 Important

- Be sure to perform the setting for both stations when conducting multiple device parallel tests between stations.
 - `HL_NORMAL_START` and `HL_MANUAL_START` cannot be specified simultaneously.
 - This function cannot be executed when the test status is `Testing`.
-

3. 2. 2 AT_set_prober_request()

Sets the start request from the prober to the shared memory.

```
int AT_set_prober_request(int stno, unsigned int status)
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int status

Specifies the start request data.

Specify using the following mnemonics. Multiple statuses can be set simultaneously by delimiting them with a vertical bar (|).

Mnemonic	Request content
PB_NO_REQUEST	No request
PB_NORMAL_START	Normal start
PB_MANUAL_START	Manual start
PB_CARRIER_END	Carrier end
PB_WAFER_END	Wafer end
PB_WAFER_COUNT	Wafer count
PB_FIRST_CHIP	First chip
PB_FIRST_WAFER	First wafer
PB_CAMERA_SETUP	Camera setup

Description

This function sets the start request sent from the prober indicated by *status* to the station specified by *stno*.

The tester controller uses this information when controlling the test program.

The most recently set information is retained until the information is reset by using the AT_set_handler_request function (refer to [3. 2. 1 "AT_set_handler_request\(\)" on page 3-5](#)) or AT_set_prober_request function (refer to [3. 2. 2 "AT_set_prober_request\(\)" on page 3-7](#)). In addition, the information set by the AT_set_test_dut function (refer to [3. 3. 7 "AT_set_test_dut\(\)" on page 3-19](#)) is cleared.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

This information can be read out with a READ PROBER STATUS statement.

This statement is valid only when the PROBER is described in the socket program.

 Important

- Be sure to perform the setting for both stations when conducting multiple device parallel tests by using two stations.
 - PB_NORMAL_START and PB_MANUAL_START cannot be specified simultaneously.
 - This function cannot be executed when the test status is Testing.
-

3. 2. 3 AT_set_prober_address()

Sets the address (XY coordinator values) of DUT1 to the shared memory.

```
int AT_set_prober_address(int stno, short x-addr, short y-addr);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

short x-addr

Specifies the X address.

Specify a value of -99 to 999 by using a variable or constant.

short y-addr

Specifies the Y address.

Specify a value of -99 to 999 by using a variable or constant.

Description

This function sets the DUT1 address indicated by *x-addr* and *y-addr* to the station specified by *stno*.

When the AT_set_prober_request function (refer to [3. 2. 2 "AT_set_prober_request\(\)" on page 3-7](#)) is executed, this setting information is cleared.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

This information can be read out with a READ PROBER ADDRESS statement.

This statement is valid only when the PROBER is described in the socket program.

! Important

This function cannot be executed when the test status is Testing.

3. 2. 4 AT_set_prober_location()

Sets the location data to the shared memory.

```
int AT_set_prober_location(int stno, unsigned int loc);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int loc

Specifies the location data.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the location data indicated by *loc* to the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

This information can be read out with a READ PROBER LOCATION statement.

This statement is valid only when the PROBER is described in the socket program.

! Important

This function cannot be executed when the test status is Testing.

3. 3 Initial Setting Library

3. 3. 1 AT_set_proname()

Sets the test program name to the shared memory.

```
int AT_set_proname(int stno, const char *name);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *name

Specifies the pointer to the area where the test program name is stored.

The test program name must be specified with a character string and terminated with a NULL code.

When the tester controller is in the hybrid mode:

Specify the pointer using a string of 128 characters or less including the path name. However, the file name must be 98 characters or less.

When the tester controller is not in the hybrid mode:

Specify the file name with a string of 32 or less lowercase alphanumeric characters including underscores (_) and dollar signs (\$), plus an extension of three characters. However, you cannot specify an extension other than pro. In addition, you cannot specify the path name.

Programs existing in the current work directory or DP 0 can be specified for execution.

➔ For more information on the Hybrid mode, refer to [ASX/U-51 Hybrid Controller User's Manual\(No. : 8409598\)](#).

Description

This function sets the test program name indicated by *name* to the station specified by *stno*.

Execution of the AT_start_program function (refer to [3. 4. 1 "AT_start_program\(\)" on page 3-39](#)) causes the test program specified by this function to be executed.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

ⓘ Important

This function cannot be executed when the test status is Testing.

3. 3. 2 AT_set_test_mode()

Sets the test mode to the shared memory.

```
int AT_set_test_mode(int stno, unsigned short mode);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned short mode

Specifies the test mode data.

Specify using the following mnemonics. Multiple modes can be specified by delimiting them with a vertical bar (|).

Test mode	Content
TEST_MODE_AUTO	Auto
TEST_MODE_MANUAL	Manual
TEST_MODE_RECYCLE	Recycle
TEST_MODE_HOLD	Hold
TEST_MODE_DISABLE	Disable

PG operation mode	Content
PG_MODE_FAIL_STOP	Fail stop
PG_MODE_FAIL_PAUSE	Fail pause
PG_MODE_FAIL_INHI	Fail inhibit
PG_MODE_REPEAT	Repeat

Description

This function sets the test mode and PG operation mode indicated by *mode* to the station specified by *stno*.

The tester controller uses this information to control the test program.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- The specification of TEST_MODE_AUTO is ignored when TEST_MODE_AUTO and TEST_MODE_MANUAL are specified at the same time.
 - This function cannot be executed when the test status is Testing.
-

3. 3. 3 AT_set_fk()

Sets function keys to the shared memory.

```
int AT_set_fk(int stno, unsigned short fk);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned short fk

Specifies the function key data.

Specify a value of 0 to 65535 by using a variable or constant. Each bit is associated with a function key as shown below:

16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
----	----	----	----	----	----	----	---	---	---	---	---	---	---	---	---

Description

This function sets the function key indicated by *fk* to the station specified by *stno*.

The tester controller uses this information to control the test program.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 3. 4 AT_set_datalog_mode()

Sets the test mode to the shared memory.

```
int AT_set_datalog_mode(int stno, unsigned short mode);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned short mode

Specifies the data log mode.

Specify using the following mnemonics. Multiple statuses can be specified by delimiting them with a vertical bar (|).

Mnemonic	Content
LOG_DISABLE	Disabling the data log mode
LOG_FUNC	Function log
LOG_PARA	DC log
LOG_PASS	PASS log
LOG_FAIL	FAIL log
LOG_MARK	MARK log
LOG_TEST_PIN	Test pin log
LOG_TEST_DATA	Test data log

Description

This function sets the data log mode indicated by *mode* to the station specified by *stno*.

The tester controller uses this information to control the data log.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

Important

- The specification of LOG_DISABLE is ignored if LOG_DISABLE and another mnemonic are specified at the same time.
- This function cannot be executed when the test status is Testing.

3. 3. 5 AT_set_test_number()

Sets the start test number and stop test number to the shared memory.

```
int AT_set_test_number(int stno, int start, int stop);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int start

Specifies the start test number.

Specify a value of 0 to 99999 by using a variable or constant.

int stop

Specifies the stop test number.

Specify a value of 0 to 99999 by using a variable or constant.

Description

This function sets the test numbers indicated by *start* and *stop* to the station specified by *stno*.

The tester controller uses this information to control the test program.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 3. 6 AT_set_init()

Turns on or off the INIT or CAL INIT lamp of the CBOX.

```
int AT_set_init(int stno, unsigned int mode);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int mode

Specifies the lamp to be turned on.

Specify using the following mnemonics. Multiple statuses can be specified by delimiting them with a vertical bar (|).

Mnemonic	Content
CBOX_DISABLE	Turning off the INIT or CAL INIT lamp
CBOX_INIT	Turning on the INIT lamp
CBOX_CAL_INIT	Turning on the CAL INIT lamp

Description

This function turns on or off the lamp specified by *mode* from among those of the CBOX connected the station specified by *stno*.

The CAL_INIT lamp is turned off when only CBOX_INIT is specified. The INIT lamp is also turned off when only CBOX_CAL_INIT is specified.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- The specification of CBOX_DISABLE is ignored if CBOX_DISABLE, CBOX_INIT, and CBOX_CAL_INIT are specified at the same time.
- This function cannot be executed when the test status is Testing.

3. 3. 7 AT_set_test_dut()

Sets the test target devices in units of 32 DUTs to the shared memory.

```
int AT_set_test_dut(int stno, int n, unsigned int *sdut);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

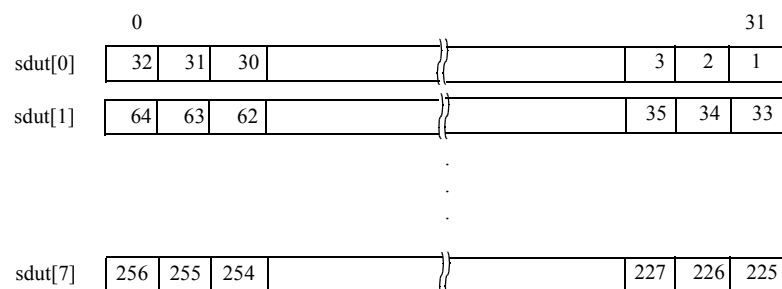
Specifies the number of array elements.

Specify a value of 1 to 8 by using a variable or constant.

unsigned int *sdut

Pointer for the target measurement device area.

Each DUT is associated with the setting data as follows:



Description

This function sets the test target device information for n devices in the area pointed to by the pointer variable *sdut* to the station specified by *stno*.

The tester controller uses this information to control the test program.

This information can be read out with a READ SDUT statement. This information can also be read out with a READ PROBER ONWAFER statement when a prober is being used.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 3. 8 AT_set_DEVICE()

Sets the system variable DEVICE.

```
int AT_set_DEVICE(int stno, const char *device);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *device

Specifies the DEVICE variable data.

The data to be set must be specified with a string of 8 characters or less and terminated with a NULL code. All characters from the 9th character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *device* to the DEVICE variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3. 9 AT_set_LOT()

Sets the system variable LOT.

```
int AT_set_LOT(int stno, const char *lot);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *lot

Specifies the LOT variable data.

The data to be set must be specified with a string of 24 characters or less and terminated with a NULL code. All characters from the 25th character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *lot* to the LOT variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.10 AT_set_SUBLOT()

Sets the system variable SUBLOT.

```
int AT_set_SUBLOT(int stno, unsigned int subplot);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int subplot

Specifies the SUBLOT variable data.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the data pointed to by the pointer variable *subplot* to the SUBLOT variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.11 AT_set_DUTIF()

Sets the system variable DUTIF.

```
int AT_set_DUTIF(int stno, const char *dutf);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *dutf

Specifies the DUTIF variable data.

The data to be set must be specified with a string of 12 characters or less and terminated with a NULL code. All characters from the 13th character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *dutf* to the DUTIF variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.12 AT_set_OPERAT()

Sets the system variable OPERAT.

```
int AT_set_OPERAT(int stno, const char *operat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *operat

Specifies the OPERAT variable data.

The data to be set must be specified with a string of 24 characters or less and terminated with a NULL code. All characters from the 25th character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *operat* to the OPERAT variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.13 AT_set_COMMEN()

Sets the system variable COMMEN.

```
int AT_set_COMMEN(int stno, const char *commen);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *commen

Specifies the COMMEN variable data.

The data to be set must be specified with a string of 40 characters or less and terminated with a NULL code. All characters from the 41st character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *commen* to the COMMEN variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.14 AT_set_WAFER()

Sets the system variable WAFER.

```
int AT_set_WAFER(int stno, const char *wafer);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *wafer

Specifies the WAFER variable data.

The data to be set must be specified with a string of 24 characters or less and terminated with a NULL code. All characters from the 25th character and beyond are ignored.

Description

This function sets the character string in the area pointed to by the pointer variable *wafer* to the WAFER variable of the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.15 AT_read_DEVICE()

Reads out the system variable DEVICE.

```
int AT_read_DEVICE(int stno, int n, char *device);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *device

Pointer for the DEVICE variable data storage location variable.

Description

This function reads data from the DEVICE variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *device*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.16 AT_read_LOT()

Reads out the system variable LOT.

```
int AT_read_LOT(int stno, int n, char *lot);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *lot

Pointer for the LOT variable data storage location variable.

Description

This function reads data from the LOT variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *lot*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.17 AT_read_SUBLOT()

Reads out the system variable SUBLOT.

```
int AT_read_SUBLOT(int stno, unsigned int *sublot);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int *sublot

Pointer for the SUBLOT variable data storage location variable.

Description

This function reads data from the SUBLOT variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *sublot*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.18 AT_read_DUTIF()

Reads out the system variable DUTIF.

```
int AT_read_DUTIF(int stno, int n, char *dutf);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *dutf

Pointer for the DUTIF variable data storage location variable.

Description

This function reads data from the DUTIF variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *dutf*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.19 AT_read_OPERAT()

Reads out the system variable OPERAT.

```
int AT_read_OPERAT(int stno, int n, char *operat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *operat

Pointer for the OPERAT variable data storage location variable.

Description

This function reads data from the OPERAT variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *operat*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.20 AT_read_COMMEN()

Reads out the system variable COMMEN.

```
int AT_read_COMMEN(int stno, int n, char *commen);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *commen

Pointer for the COMMEN variable data storage location variable.

Description

This function reads data from the COMMEN variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *commen*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.21 AT_read_WAFER()

Reads out the system variable WAFER.

```
int AT_read_WAFER(int stno, int n, char *wafer);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *wafer

Pointer for the WAFER variable data storage location variable.

Description

This function reads data from the WAFER variable of the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *wafer*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable data must have a length equal to the number of read out characters +1.

3. 3.22 AT_set_opdat()

Sets the operator data to the shared memory.

```
int AT_set_opdat(int stno, const char *data);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *data

Specifies the pointer to the operator data area.

The data to be set must be specified with a string of 128 characters or less and terminated with a NULL code. All characters from the 129th character and beyond are ignored.

Description

This function sets the operator data in the area pointed to by the pointer variable *data* to the station specified by *stno*.

This information can be read by the AT_read_opdat function (refer to [3. 3. 23 "AT_read_opdat\(\)" on page 3-35](#)) or the RDOPDAT utility (refer to [3. 7. 2 "RDOPDAT" on page 3-86](#)).

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 3.23 AT_read_opdat()

Reads out the operator data from the shared memory.

```
int AT_read_opdat(int stno, int n, char *data);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be read out.

char *data

Pointer for the operator data storage location variable.

Description

This function reads the operator data from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *data*.

This information can be read by the AT_set_opdat function (refer to [3. 3. 22 "AT_set_opdat\(\)" on page 3-34](#)) or the WROPDAT utility (refer to [3. 7. 1 "WROPDAT" on page 3-85](#)).

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The NULL code is appended to the end of data items. Therefore, storage location variable *data* must have a length equal to the number of read out characters +1.

3. 3.24 AT_clear_program()

Clears the program area within the shared memory.

```
int AT_clear_program(void);
```

Description

Clears the program managed in the shared memory. This function is equivalent to Mode 1 of the controller key command U.

Proper clearing results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 3.25 AT_read_pbid()

Reads out the performance board ID.

```
int AT_read_pbid(int stno, int n, unsigned int *pbid);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of data items to be read out.

Specify 1 or 2 by using a variable or constant.

unsigned int *pbid

Pointer for the performance board ID storage location variable.

Description

This function reads the ID of the performance board implemented on the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *pbid*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 3.26 AT_read_pdtype()

Reads out the type of the performance board.

```
int AT_read_pdtype(int stno, int *dutno, unsigned int *pdtype);
```

Argument: int stno

Specifies the station number.
Specify 1 or 2 by using a variable or constant.

int *dutno

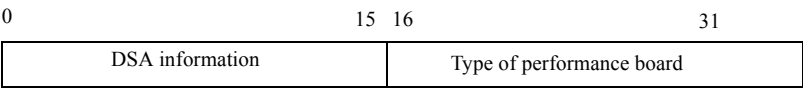
Pointer for the number of devices tested in parallel storage location variable.

unsigned int *pdtype

Pointer for the performance board type storage location variable.

Description

This function reads the PBDTYPE of the performance board implemented on the station specified by *stno* and stores the number of devices tested in parallel into the area pointed to by the pointer variable *dutno* and the PBDTYPE into the area pointed to by the pointer variable *pdtype*.
The data format of pdtype is as follows:



Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

!

Important

This function cannot be executed when the test status is Testing.

3. 4 Program Control Library

3. 4. 1 AT_start_program()

Executes the test program set by the AT_set_proname function.

```
int AT_start_program(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

This function executes the program set to the station specified by *stno*.

The program name is set by the AT_set_proname function (refer to [3. 3. 1 "AT_set_proname\(\)" on page 3-11](#)).

If the executed program is an inter-station parallel test program, the program at the associated station is started automatically.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

This function cannot be executed when the test status is Testing.

3. 4. 2 AT_reset_program()

Suspends the program forcibly.

```
int AT_reset_program(void);
```

Description

This function forcibly suspends the program in progress. This function is equivalent to [Ctrl]+[C].

Proper interruption results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 4. 3 AT_wait_program()

Waits until the program execution is terminated.

```
int AT_wait_program(void);
```

Description

This function waits until the device program ends.

Proper interruption results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

3. 4. 4 AT_read_test_status()

Reads out the test status.

```
int AT_read_test_status(int *status);
```

Argument: int *status

Pointer for the test status storage location variable.

The relationship between the read out value and the test status is shown below:

Value	Content
1	READY
2	TEST END
3	ALARM
-4	The subroutine program is being executed.
-3	The CAL program is being executed.
-2	TESTING
-1	PAUSE

Description

This function reads the test status and stores the read data into the area pointed to by the pointer variable *status*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 4. 5 AT_start_subprogram()

Executes a subroutine program.

```
int AT_start_subprogram(const char *name);
```

Argument: const char *name

Specifies the pointer to the subroutine program name area.

The subroutine program name must be specified with a character string and terminated with a NULL code.

Specify the file name with a string of 32 or less lowercase alphanumeric characters including underscores (_) and dollar signs (\$), plus an extension of three characters. However, you cannot specify an extension other than sub or lib. Additionally, path names cannot be specified.

Description

This function executes the subroutine program specified by *name*.

The subroutine program existing in the current work directory or DP0 is specified for execution.

The subroutine program is executed after the test end when the test status is Testing.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5 Measurement Results Processing Library

3. 5. 1 AT_read_fail_dut()

Reads out the failed device information.

```
int AT_read_fail_dut(int stno, int n, unsigned int *fdut);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

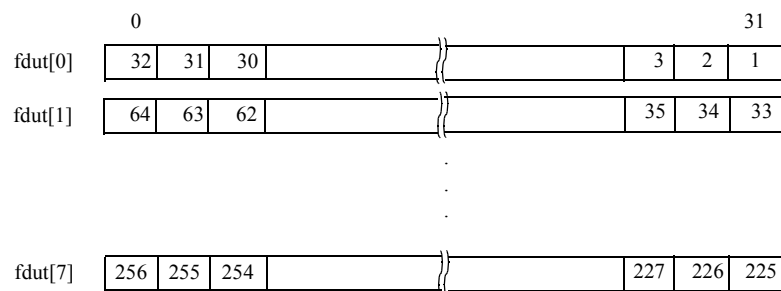
Specifies the number of data items to be read out.

Specify a value of 1 to 8 by using a variable or constant.

unsigned int *fdut

Specifies the pointer to the destination variable in which the fail device information is stored.

The relationship between the read out information and each DUT is as shown below.



Description

This function reads the failed device information from the station specified by *stno* and stores the read information into the area pointed to by the pointer variable *fdut* for *n* DUTs.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

! Important

The destination storage variable fdut must have a capacity large enough to store data for n read out data items.

3. 5. 2 AT_read_category_data()

Reads out category data.

```
int AT_read_category_data(int stno, int n, unsigned int *cat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of data items to be read out.

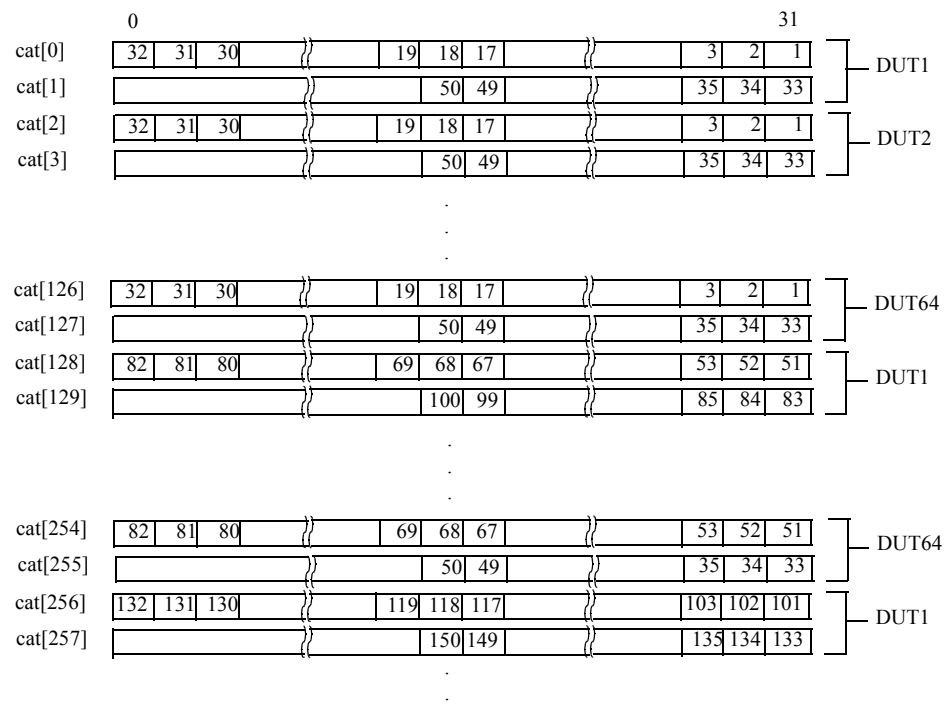
Specify a value of 1 to 10240 by using a variable or constant.

unsigned int *cat

Pointer for the category data storage location variable.

Data is read out in units of 50 categories for each DUT as shown below:

When 64 DUTs are provided for each



Description

This function reads the category data from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *cat* for *n* DUTs.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The destination storage variable *cat* must have a capacity large enough to store data for *n* read out data items.

3. 5. 3 AT_read_sort_data()

Reads out sort data.

```
int AT_read_sort_data(int stno, int n, unsigned int *sort);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of data items to be read out.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int *sort

Pointer for the sort data storage location variable.

Description

This function reads the sort data from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *sort* for *n* DUTs.

The relationship between the read out information and each DUT is as shown below.

```
sort[0] ..... DUT1 sort data
sort[1] ..... DUT2 sort data
      .
      .
sort[254] ..... DUT255 sort data
sort[255] ..... DUT256 sort data
```

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The destination storage variable *sort* must have a capacity large enough to store data for *n* read out data items.

3. 5. 4 AT_read_total_counter()

Reads out the total counter.

```
int AT_read_total_counter(int stno, int dutno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

unsigned int *counter

Pointer for the total counter storage location variable.

Description

This function reads the total counter from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5. 5 AT_read_fail_counter()

Reads out the fail counter.

```
int AT_read_fail_counter(int stno, int dutno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

unsigned int *counter

Pointer for the fail counter storage location variable.

Description

This function reads the fail counter from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5. 6 AT_read_category_counter()

Reads out the category counter.

```
int AT_read_category_counter(int stno, int dutno, int catno1, int catno2, unsigned
int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

int catno1

Specifies the category number at which reading begins.

Specify a value of 1 to 999 by using a variable or constant.

int catno2

Specifies the category number at which reading ends.

Specify a value of 1 to 999 by using a variable or constant.

unsigned int *counter

Pointer for the category counter storage location variable.

Description

This function reads the category counter from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

The category counter is read continuously from *catno1* to *catno2*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5. 7 AT_read_sort_counter()

Reads out the sort counter.

```
int AT_read_sort_counter(int stno, int dutno, int sortno1, int sortno2, unsigned
int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

int sortno1

Specifies the sort number at which reading begins.

Specify a value of 0 to 16 by using a variable or constant.

int sortno2

Specifies the sort number at which reading ends.

Specify a value of 0 to 16 by using a variable or constant.

unsigned int *counter

Pointer for the sort counter storage location variable.

Description

This function reads the sort counter from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

The sort counter is read continuously from *sortno1* to *sortno2*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*. This function reads out the sort total counter when 0 is specified for *sortno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5. 8 AT_read_test_fail_counter()

Reads out the test fail counter.

```
int AT_read_test_fail_counter(int stno, int testno1, int testno2, unsigned int
*counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int testno1

Specifies the test number at which reading begins.

Specify a value of 0 to 9999 by using a variable or constant.

int testno2

Specifies the test number at which reading ends.

Specify a value of 0 to 9999 by using a variable or constant.

unsigned int *counter

Pointer for the test fail counter storage location variable.

Description

This function reads the test fail counter from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

If reading is performed to the counter for a test number which is not defined in the controller key command ".T", the counter value is 0.

The test fail counter is read continuously from *testno1* to *testno2*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

3. 5. 9 AT_read_sample_number()

Reads out a sample number.

```
int AT_read_sample_number(int stno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int *counter

Pointer for the sample number storage location variable.

Description

This function reads a sample number from the station specified by *stno* and stores the read number into the area pointed to by the pointer variable *counter*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.10 AT_read_wafer_total_counter()

Reads out the total counter on the wafer.

```
int AT_read_wafer_total_counter(int stno, int dutno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

unsigned int *counter

Pointer for the total counter on the wafer storage location variable.

Description

This function reads the total counter on the wafer from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.11 AT_read_wafer_fail_counter()

Reads out the fail counter on the wafer.

```
int AT_read_wafer_fail_counter(int stno, int dutno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

unsigned int *counter

Pointer for the fail counter on the wafer storage location variable.

Description

This function reads the fail counter on the wafer from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.12 AT_read_wafer_category_counter()

Reads out the category counter on the wafer.

```
int AT_read_wafer_category_counter(int stno, int dutno, int catno, unsigned int
*counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number for which data is read out.

Specify a value of 0 to 256 by using a variable or constant.

int catno

Specifies the category number for which data is read out.

Specify a value of 1 to 999 by using a variable or constant.

unsigned int *counter

Pointer for the category counter on the wafer storage location variable.

Description

This function reads the category counter on the wafer from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

This function reads the grand total for all DUTs when 0 is specified for *dutno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.13 AT_read_wafer_counter()

Reads out the number of wafers.

```
int AT_read_wafer_counter(int stno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int *counter

Pointer for the number of wafers storage location variable.

Description

This function reads the number of wafers from the station specified by *stno* and stores the read data into the area pointed to by the pointer variable *counter*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.14 AT_read_slice_number()

Reads out a slice number.

```
int AT_read_slice_number(int stno, unsigned int *counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int *counter

Pointer for the slice number storage location variable.

Description

This function reads a slice number from the station specified by *stno* and stores the read number into the area pointed to by the pointer variable *counter*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.15 AT_set_total_counter()

Sets the total counter.

```
int AT_set_total_counter(int stno, int dutno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the total counter.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the total counter for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.16 AT_set_fail_counter()

Sets the fail counter.

```
int AT_set_fail_counter(int stno, int dutno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the fail counter.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the fail counter for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.17 AT_set_category_counter()

Sets the category counter.

```
int AT_set_category_counter(int stno, int dutno, int catno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

int catno

Specifies the category number to be set.

Specify a value of 1 to 999 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the category counter.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the category counter for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.18 AT_set_sort_counter()

Sets the sort counter.

```
int AT_set_sort_counter(int stno, int dutno, int sortno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

int sortno

Specifies the sort number to be set.

Specify a value of 0 to 16 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the sort counter.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the sort counter for the station specified by *stno*.

This function sets the sort total counter when 0 is specified for *sortno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.19 AT_set_test_fail_counter()

Sets the test fail counter.

```
int AT_set_test_fail_counter(int stno, int testno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int testno

Specifies the test number for which the test fail counter is set.

Specify a value of 0 to 9999 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the test fail counter.

Specify a value of 0 to 65535 (16 bits) by using a variable or constant.

Description

This function sets the test fail counter for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Test numbers must be defined in advance by the controller key command ".T".

3. 5.20 AT_set_sample_number()

Sets a sample number.

```
int AT_set_sample_number(int stno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int counter

Specifies the value to be set as the sample number.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the sample number for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.21 AT_set_wafer_total_counter()

Sets the total counter on the wafer.

```
int AT_set_wafer_total_counter(int stno, int dutno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the total counter on the wafer.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the total counter on the wafer for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.22 AT_set_wafer_fail_counter()

Sets the fail counter on the wafer.

```
int AT_set_wafer_fail_counter(int stno, int dutno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the fail counter on the wafer.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the fail counter on the wafer for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.23 AT_set_wafer_category_counter()

Sets the category counter on the wafer.

```
int AT_set_wafer_category_counter(int stno, int dutno, int catno, unsigned int
counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be set.

Specify a value of 1 to 256 by using a variable or constant.

int catno

Specifies the category number to be set.

Specify a value of 1 to 999 by using a variable or constant.

unsigned int counter

Specifies the value to be set to the category counter on the wafer.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the category counter on the wafer for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.24 AT_set_wafer_counter()

Sets the number of wafers.

```
int AT_set_wafer_counter(int stno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int counter

Specifies the value to be set as the number of wafers.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the number of wafers for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.25 AT_set_slice_number()

Sets a slice number.

```
int AT_set_slice_number(int stno, unsigned int counter);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int counter

Specifies the value to be set as the slice number.

Specify a value of 0 to 4294967295 (32 bits) by using a variable or constant.

Description

This function sets the slice number for the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.26 AT_clear_total_counter()

Sets the total counter to 0.

```
int AT_clear_total_counter(int stno, int dutno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

Description

This function sets the total counter for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.27 AT_clear_fail_counter()

Sets the fail counter to 0.

```
int AT_clear_fail_counter(int stno, int dutno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

Description

This function sets the fail counter for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.28 AT_clear_category_counter()

Sets the category counter to 0.

```
int AT_clear_category_counter(int stno, int dutno, int catno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

int catno

Specifies the category number to be cleared.

Specify a value of 1 to 999 by using a variable or constant.

Description

This function sets the category counter for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.29 AT_clear_sort_counter()

Sets the sort counter to 0.

```
int AT_clear_sort_counter(int stno, int dutno, int sortno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

int sortno

Specifies the sort number for which the sort counter is cleared.

Specify a value of 0 to 16 by using a variable or constant.

Description

This function sets the sort counter for the station specified by *stno* to 0.

This function sets the sort total counter to 0 when 0 is specified for *sortno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.30 AT_clear_test_fail_counter()

Sets the test fail counter to 0.

```
int AT_clear_test_fail_counter(int stno, int testno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int testno

Specifies the test number for which data is read out.

Specify a value of 0 to 9999 by using a variable or constant.

Description

This function sets the test fail counter for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Test numbers must be defined in advance by the controller key command ".T".

3. 5.31 AT_clear_sample_number()

Sets the sample number to 0.

```
int AT_clear_sample_number(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

This function sets the sample number for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.32 AT_clear_wafer_total_counter()

Sets the total counter on the wafer to 0.

```
int AT_clear_wafer_total_counter(int stno, int dutno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

Description

This function sets the total counter on the wafer for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.33 AT_clear_wafer_fail_counter()

Sets the fail counter on the wafer to 0.

```
int AT_clear_wafer_fail_counter(int stno, int dutno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

Description

This function sets the fail counter on the wafer for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.34 AT_clear_wafer_category_counter()

Sets the category counter on the wafer to 0.

```
int AT_clear_wafer_category_counter(int stno, int dutno, int catno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the DUT number to be cleared.

Specify a value of 1 to 256 by using a variable or constant.

int catno

Specifies the category number to be cleared.

Specify a value of 1 to 999 by using a variable or constant.

Description

This function sets the category counter on the wafer for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.35 AT_clear_wafer_counter()

Sets the number of wafers to 1.

```
int AT_clear_wafer_counter(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

This function sets the number of wafers for the station specified by *stno* to 1.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 5.36 AT_clear_slice_number()

Sets the slice number to 0.

```
int AT_clear_slice_number(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

This function sets the slice number for the station specified by *stno* to 0.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 6 Other Functions

3. 6. 1 AT_set_lock()

Locks an application.

```
int AT_set_lock(int n);
```

Argument: int n

Specifies the sequence number.

Specify a value of 0 to 99 by using a variable or constant.

Description

This function locks the application that is using the sequence number *n*. The obtained lock is not reset until the AT_reset_loc function is executed.

Combined use of this function and the AT_reset_loc function (refer to [3. 6. 2 "AT_reset_lock\(\)" on page 3-84](#)) prohibits multiple starting of a program.

Proper execution results in a normal end (Error Code = 0). However, if a lock is already applied or any exception or abnormality exists returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Even a different program cannot be executed if it is using the same sequence number.
- Be sure to reset the lock before the program ends because the lock information is held even after the program ends. If a program is not unlocked before it ends, the program cannot be restarted.

Sample Program

- (1) While the Following Program is Operating, Start from Another Process is Disabled.

```
#include "AT/testerlib.h"
main() {
    if (AT_set_lock(0)) exit(1);
    .
    .
    AT_reset_lock(0);
}
```

3. 6. 2 AT_reset_lock()

Unlocks an application.

```
int AT_reset_lock(int n);
```

Argument: int n

Specifies the sequence number.

Specify a value of 0 to 99 by using a variable or constant.

Description

This function unlocks the application that is using the sequence number n .

Combined use of this function and the AT_set_loc function (refer to [3. 6. 1 "AT_set_lock\(\)" on page 3-82](#)) prohibits multiple starting of a program.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Even a different program cannot be executed if it is using the same sequence number.
- Be sure to reset the lock before the program ends because the lock information is held even after the program ends. If a program is not unlocked before it ends, the program cannot be restarted.

Sample Program

(1) While the Following Program is Operating, Start from Another Process is Disabled.

```
#include "AT/testerlib.h"
main() {
  if (AT_set_lock(0)) exit(1);
  .
  .
  AT_reset_lock(0);
}
```

3. 7 Tester Utilities

Tester utilities have been provided with functions equivalent to the `AT_set_opdat()` and `AT_read_opdat()` functions. Use of these utilities allows read and write of operation data from the test program as well.

3. 7. 1 WROPDAT

Function: This utility is used to set operation data to the shared memory.

Syntax

```
CALL WROPDAT(arg1, arg2, arg3)
```

Argument: arg1

Specifies the station number.

Specify using a double-word int-type variable or constant having a value within the range of 0 to 2.

arg2

Specifies the operator data area.

Specify using all single-word char-type array elements.

arg3

Specifies the error code storage location variable.

Specify using a double-word int-type variable.

Description

This utility sets the operator data in the area specified by *arg2* to the station specified by *arg1*. All characters from the 129th character and beyond are ignored.

The operation data is set to the station where the program is now being executed, if 0 is set to *arg1*.

This information can be read by the `AT_read_opdat` function (refer to [3. 3. 23 "AT_read_opdat\(\)" on page 3-35](#)) or the `RDOPDAT` utility (refer to [3. 7. 2 "RDOPDAT" on page 3-86](#)).

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

3. 7. 2 RDOPDAT

Function: This utility is used to read out the operator data from the shared memory.

Syntax

```
CALL RDOPDAT(arg1, arg2, arg3)
```

Argument: arg1

Specifies the station number.

Specify using a double-word int-type variable or constant having a value within the range of 0 to 2.

arg2

Specifies the destination area into which the operator data is stored.

Specify using all single-word char-type array elements.

arg3

Specifies the error code storage location variable.

Specify using a double-word int-type variable.

Description

This utility reads the operator data from the station specified by *arg1* and stores the read data into the area specified by *arg2*. If the operator data is greater than *arg2*, only the portion of the operator data as large as *arg2* is read out.

The operation data is read out of the station where the program is now being executed, if 0 is set to *arg1*.

This information can be set by the `AT_set_opdat` function (refer to [3. 3. 22 "AT_set_opdat\(\)" on page 3-34](#)) or the `CALL WROPDAT` utility (refer to [3. 7. 1 "WROPDAT" on page 3-85](#)).

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

4. GPIB Control Library

The GPIB control library contains functions used to control the HANDLER and PROBER through the GPIB.

4. 1 Overview

The GPIB library is separated into the following three groups of functions. This section describes overviews of individual function groups.

- Functions for controlling GPIB
- Functions for controlling the PROBER
- Functions for controlling the HANDLER

4. 1. 1 Functions for Controlling GPIB

A group of functions to give or receive data through the GPIB interface.

Table 4-1 List of Functions for Controlling GPIB

Function name	Function
AT_set_GPIB_address()	Sets the GPIB address
AT_set_GPIB_timeout()	Sets the GPIB timeout
AT_send_GPIB_data()	Transfers the GPIB data
AT_read_GPIB_data()	Receives the GPIB data

4. 1. 2 Functions for Controlling the PROBER

A group of functions to control the PROBER using the GPIB.

Table 4-2 List of Functions for Controlling the PROBER

Function name	Function
AT_select_prober()	Selects the prober
AT_read_prober_machineID()	Acquires the machine number
AT_read_prober_lotname()	Acquires the lot name
AT_read_prober_waferID()	Acquires the wafer ID
AT_read_prober_location()	Acquires the location

Function name	Function
AT_read_prober_address()	Acquires the XY address
AT_read_prober_onwafer()	Acquires the on-wafer information
AT_set_prober_Zup()	Rises the Z-axis
AT_set_prober_Zdown()	Falls the Z-axis
AT_start_prober_test()	Starts the test
AT_send_prober_test_result()	Outputs the test result
AT_set_prober_next_die()	Shifts to the next die

4. 1. 3 Functions for Controlling the HANDLER

A group of functions to control the HANDLER using the GPIB.

Table 4-3 List of Functions for Controlling the HANDLER

Function name	Function
AT_read_handler_status()	Acquires the operational status
AT_set_handler_SRQMASK()	Sets the SRQMASK
AT_read_handler_SRQKIND()	Acquires the special processing SRQ factor
AT_set_handler_SRQ_answer()	Returns the reply to the SRQ
AT_read_handler_devicename()	Acquires the device name
AT_read_handler_lotname()	Acquires the lot name
AT_set_handler_DUTNUMSEL()	Sets the DUTNUMSEL data
AT_read_handler_DUTNUMSEL()	Acquires the DUTNUMSEL data
AT_read_handler_temp()	Acquires the temperature
AT_set_handler_PBTYPY()	Sets the PBTYPY
AT_set_handler_TESTCNT()	Sets the TESTCNT
AT_read_handler_retest_category()	Acquires the re-test category
AT_read_handler_DOUBLETEST_category()	Acquires the DOUBLETEST category
AT_set_handler_TESTALARM()	Interrupts the test
AT_read_handler_testdut()	Acquires the device set status
AT_set_handler_sort()	Sets the sort category
AT_clear_handler_retest_sort()	Clears the re-test sort category counter

4. 2 Functions for Controlling GPIB

A group of functions to give or receive data through the GPIB interface

4. 2. 1 AT_set_GPIB_address()

Sets the GPIB address.

```
int AT_set_GPIB_address(int stno, int address);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int address

Specifies the GPIB address.

Specify using a variable or constant with a value of -1 or 1 to 30.

Description

Sets the GPIB address shown by *address* to the station specified by *stno*. If this function is not executed, the GPIB cannot be used.

If -1 is specified for *address*, the GPIB of the station is released.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

4. 2. 2 AT_set_GPIB_timeout()

Sets the timeout time of the GPIB.

```
int AT_set_GPIB_timeout(int stno, int time);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int time

Specifies the timeout time.

Specify using the following mnemonics.

Mnemonic	Timeout time
TNONE	Not specified
T10US	10 uS
T30US	30 uS
T100US	100 uS
T300US	300 uS
T1MS	1 mS
T3MS	3 mS
T10MS	10 mS
T30MS	30 mS
T100MS	100 mS
T300MS	300 mS
T1S	1 S
T3S	3 S
T10S	10 S
T30S	30 S
T100S	100 S

Description

The GPIB timeout time of the station specified by *stno* is set to the time specified by *time*.

If this function is not executed, the timeout time is set to 10 seconds.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

4. 2. 3 AT_send_GPIB_data()

Transmits the GPIB data.

```
int AT_send_GPIB_data(int stno, const char *data);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

const char *data

Pointer for the transmitting data area.

Description

The <CR><LF> code is added to the data in the area shown by the pointer variable *data* and set to the GPIB unit of the station specified by *stno*.

Data to be transmitted must end with the NULL code.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 2. 4 AT_read_GPIB_data()

Receives the GPIB data.

```
int AT_read_GPIB_data(int stno, int n, char *data);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

Specify with a variable or constant.

char *data

Pointer for the received data storage location variable.

Description

Receives data from the GPIB unit of the station specified by *stno* and stores *n* characters in the area shown by the pointer variable *data*.

The receiving process is completed when <EOI> is received. Received data is stored after deleting the <CR><LF> code.

Proper reception results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- The NULL code is appended to the end of data items. Therefore, storage location variable *data* must have a length equal to the number of read out characters +1.
- Before executing this function, the `AT_set_GPIB_address` function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 2. 5 AT_read_GPIB_status_byte()

Acquires the status byte data of the GPIB.

```
int AT_read_GPIB_status_byte(int *stb);
```

Argument: int *stb

Pointer for the status byte storage location variable.

Description

Performs serial polling to the address specified by the AT_set_GPIB_address function and receives status byte data. Received status byte data is stored in the area shown by the pointer variable *stb*. The timeout time for serial polling is 100 MS regardless of the setting of the AT_set_GPIB_timeout function (refer to [4. 2. 2 "AT_set_GPIB_timeout\(\)" on page 4-4](#)).

Data will be stored as follows:

stb[0] : Information on the unit connected to station 1
stb[1] : Information on the unit connected to station 2

When the SRQ has not occurred or the GPIB address is not set in the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)), 0 will be stored and, if any error occurs, -1 will be stored.

Proper reception results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

The status byte data storage area must be an array variable with two or more elements.

4. 3 Functions for Controlling the PROBER

A group of functions to control the PROBER using the GPIB.

4. 3. 1 AT_select_prober()

Selects the connected probers.

```
int AT_select_prober(int stno, int sel);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int sel

Specifies the prober.

Specify using the following mnemonics.

Mnemonic	Type of PROBER
PB_TEL	PROBER made by TEL
PB_TSK	PROBER made by TSK

Description

Selects the probers connected to the station specified by *stno*. If this function is not executed, the PROBER cannot be used.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

4. 3. 2 AT_read_prober_machineID()

Acquires the machine number from the prober.

```
int AT_read_prober_machineID(int stno, int n, char *machine-id);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *machine-id

Pointer for the machine number storage location variable.

Description

Acquires the machine number data from the prober connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *machine-id*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- If PB_TSK is selected in the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)), stores the data after deleting the command name "B."

4. 3. 3 AT_read_prober_lotname()

Acquires the lot name from the prober.

```
int AT_read_prober_lotname(int stno, int n, char *lotname);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *lotname

Pointer for the lot name storage location variable.

Description

Acquires the lot name from the prober connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *lotname*. Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- If PB_TSK is selected in the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)), stores the data after deleting the command name "V."

4. 3. 4 AT_read_prober_waferID()

Acquires the wafer ID from the prober.

```
int AT_read_prober_waferID(int stno, int n, char *wafer-id);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *wafer-id

Pointer for the wafer ID storage location variable.

Description

Acquires the wafer ID from the prober connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *wafer-id*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- If PB_TSK is selected in the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)), stores the data after deleting the command name "b."

4. 3. 5 AT_read_prober_location()

Acquires the location data from the XY coordinator.

```
int AT_read_prober_location(int stno, int n, char *location);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *location

Pointer for the location data storage location variable.

Description

Acquires the location data from the prober (XY coordinator) connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *location*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- If PB_TSK is selected in the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)), stores the data after deleting the command name "b."

4. 3. 6 AT_read_prober_address()

Acquires the XY address from the XY coordinator.

```
int AT_read_prober_address(int stno, int *x-addr, int *y-addr);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int *x-addr

Pointer for the X address storage location variable.

int *y-addr

Pointer for the Y address storage location variable.

Description

Acquires the DUT1 address data from the prober (XY coordinator) connected to the station specified by *stno* and stores it to the area shown by the pointer variables *x-addr* and *y-addr*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.

4. 3. 7 AT_read_prober_onwafer()

Acquires the XY address from the XY coordinator.

```
int AT_read_prober_onwafer(int stno, int n, unsigned int *onwafer);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements.

Specify a value of 1 to 8 by using a variable or constant.

unsigned int *onwafer

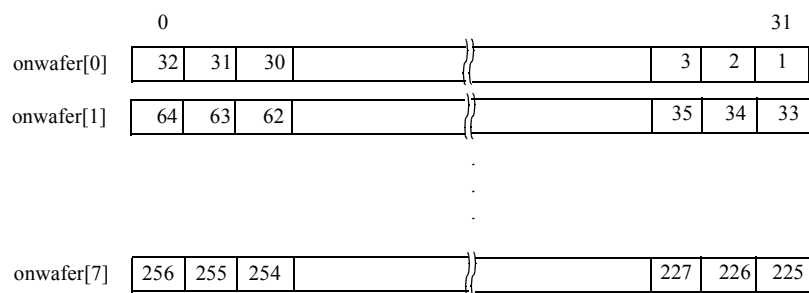
Pointer for the onwafer data storage location variable.

Description

Acquires onwafer data from the prober connected to the station specified by *stno* and stores *n* pieces in the area shown by the pointer variable *onwafer*.

If onwafer information is smaller than the number of array elements *n*, 0 will be stored in the remaining areas.

The relationship between DUTs and onwafer data is as follows:



Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

 Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.

4. 3. 8 AT_set_prober_Zup()

Raises the Z axis.

```
int AT_set_prober_Zup(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Raises the Z axis of the probers connected to the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.

4. 3. 9 AT_set_prober_Zdown()

Lowers the Z axis.

```
int AT_set_prober_Zdown(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Lowers the Z axis of the probers connected to the station specified by *stno*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.

4. 3.10 AT_start_prober_test()

Starts the test.

```
int AT_start_prober_test(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Releases the bus of the prober connected to the station specified by *stno* and starts the test.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.
- Nothing will be conducted if PB_TSK is selected by the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)).

4. 3.11 AT_send_prober_test_result()

Outputs the test result.

```
int AT_send_prober_test_result(int stno, int dutno, unsigned int *fdut, unsigned
int *cat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int dutno

Specifies the number of measured devices.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int *fdut

Pointer for the fail DUT data area.

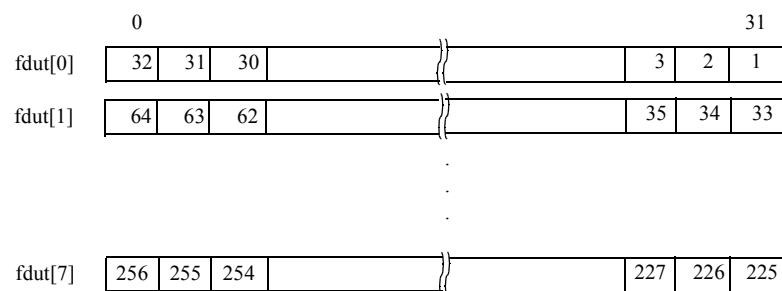
unsigned int *cat

Pointer for the category data area.

Description

Assuming that data stored in the area shown by the pointer variable *fdut* is fail DUT and the data stored in the area shown by the pointer variable *cat* is category data, transmits PASS/FAIL and category information to the prober connected to the station specified by *stno*.

The format of the fail DUT is as follows:



➔ For more information on the category format, refer to

[3. 5. 2 "AT_read_category_data\(\)" on page 3-46.](#)

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1.](#)

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
 - Does not wait for SRQ.
-

4. 3.12 AT_set_prober_next_die()

Moves the chuck to the next die.

```
int AT_set_prober_next_die(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Raises the chuck of the prober connected to the station specified by *stno* to the next die.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.

4. 3.13 AT_set_prober_target_die()

Moves the chuck to the target die.

```
int AT_set_prober_target_die(int stno, int x-addr, int y-addr);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int x-addr

Specifies the X address.

If PB_TEL is selected by the AT_select_prober function, specify a value of 0 to 999 by using a variable or constant. If PB_TSK is selected, specify a value of -99 to 511 by using a variable or constant.

int y-addr

Specifies the Y address.

If PB_TEL is selected by the AT_select_prober function, specify a value of 0 to 999 by using a variable or constant. If PB_TSK is selected, specify a value of -99 to 511 by using a variable or constant.

Description

Moves the chuck of the prober connected to the station specified by *stno* to the die shown by *x-addr* and *y-addr*.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.

4. 3.14 AT_set_prober_wafer_unload()

Unloads the wafer on the chuck and loads the next wafer.

```
int AT_set_prober_wafer_unload(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Unloads the wafer on the chuck of the prober connected to the station specified by *stno* and loads the next wafer.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) and the AT_select_prober function (refer to [4. 3. 1 "AT_select_prober\(\)" on page 4-9](#)) must have been executed.
- Does not wait for SRQ.

4. 4 Functions for Controlling the HANDLER

A group of functions to control the HANDLER using the GPIB.

4. 4. 1 AT_read_handler_status()

Acquires the operational status of the handler.

```
int AT_read_handler_status(int stno, int *status);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int *status

Pointer for the operational status storage location variable.

Description

Acquires the operational status of the handler connected to the station specified by *stno* and stores to the area shown by the pointer variable *status*.

If the handler is in the following condition, the read data is 0 and, in other cases, 1.

- Operation stopped
- Alarm has occurred
- At the lot end
- When not in temp. ready in
- When the loader is empty and the final test of the lot has been completed

Proper acquisition results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the `AT_set_GPIB_address` function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 2 AT_set_handler_SRQMASK()

Sets the SRQ mask data to the handler.

```
int AT_set_handler_SRQMASK(int stno, unsigned int mask);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int mask

Specifies the mask data.

Specify using the following mnemonics.

Mnemonic	Content of mask
HL_NO_MASK	Releasing the mask
HL_NORMAL_START_MASK	Normal start mask
HL_MANUAL_START_MASK	Manual start mask
HL_SPECIAL_SRQ_MASK	Special handling mask Test head request Lot start Re-test lot start Lot end Final lot end Tray ID read request DUT number automatic setting request Double test request

Description

Sets the mask data of the SRQ specified by *mask* to the handler connected to the station specified by *stno*.

Proper setting results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 3 AT_read_handler_SRQKIND()

Acquires the factor of the special handling SRQ.

```
int AT_read_handler_SRQKIND(int stno, unsigned int *srq);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

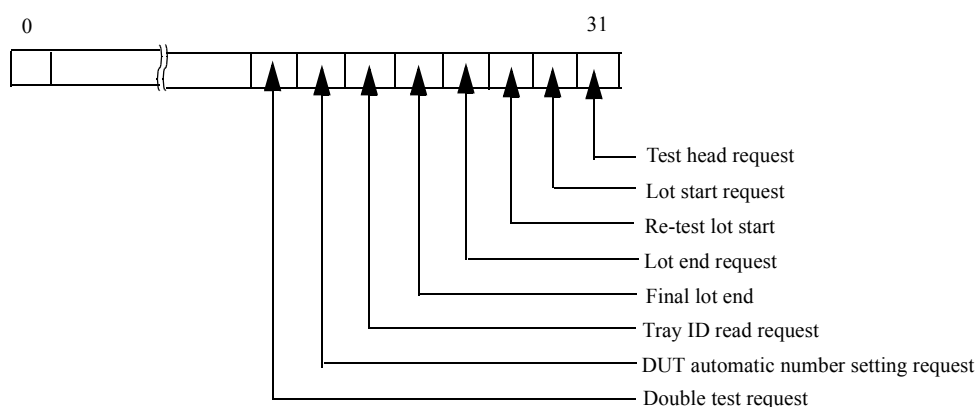
unsigned int *srq

Pointer for the special handling SRQ factor storage location variable.

Description

Acquires the special handling SRQ factor of the handler connected to the station specified by *stno* and stores to the area shown by the pointer variable *srq*.

Each factor corresponds to a bit and the details are as follows:



Proper acquisition results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the `AT_set_GPIB_address` function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 4 AT_set_handler_SRQ_answer()

Transmits the answer for the SRQ to the handler.

```
int AT_set_handler_SRQ_answer(int stno, unsigned int answer);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int answer

Specifies the answer data.

Specify using the following mnemonics.

Mnemonic	Content of answer
HL_TEST_NG_ANS	Test start not allowed
HL_LOT_START_ANS	Lot start confirmation Re-test lot start confirmation
HL_LOT_END_ANS	Lot end confirmation Final lot end confirmation

Description

Transmits the answer to the SRQ specified by *answer* to the handler connected to the station specified by *stno*.

Proper acquisition results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 5 AT_read_handler_devicename()

Acquires the device name from the handler.

```
int AT_read_handler_devicename(int stno, int n, char *devname);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *devname

Pointer for the device name storage location variable.

Description

Acquires the device name from the handler connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *devname*. Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 6 AT_read_handler_lotname()

Acquires the lot name from the handler.

```
int AT_read_handler_lotname(int stno, int n, char *lotname);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of characters to be stored.

char *lotname

Pointer for the lot name storage location variable.

Description

Acquires the lot name from the handler connected to the station specified by *stno*, and stores it for the number of characters specified by *n* to the area shown by the pointer variable *lotname*. Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

- Because the NULL code is added at the end of data, the storage location variable must have a size of the number of stored characters + 1 or more.
- Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 7 AT_set_handler_DUTNUMSEL()

Sets the DUTNUMSEL data to the handler.

```
int AT_set_handler_DUTNUMSEL(int stno, int n, char *sel);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements.

Specify a value of 1 to 256 by using a variable or constant.

char *sel

Pointer for the DUTNUMSEL data area.

Description

Sets the DUTNUMSEL data shown by the pointer variable *sel* for *n* pieces to the handler connected to the station specified by *stno*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 8 AT_read_handler_DUTNUMSEL()

Acquires the DUTNUMSEL data from the handler.

```
int AT_read_handler_DUTNUMSEL(int stno, int n, char *sel);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements.

Specify a value of 1 to 256 by using a variable or constant.

char *sel

Pointer for the DUTNUMSEL data storage location variable.

Description

Acquires the DUTNUMSEL data from the handler connected to the station specified by *stno* and stores *n* pieces in the area shown by the pointer variable *sel*.

If DUTNUMSEL information is smaller than the number of array elements *n*, 0 will be stored in the remaining areas.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4. 9 AT_read_handler_temp()

Acquires the temperature data from the handler.

```
int AT_read_handler_temp(int stno, int kind, double *temp);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int kind

Specifies the temperature control information to be read out.

Specify using the following mnemonics.

Mnemonic	Temperature to be acquired
HL_SETTEMP	Set temperature
HL_MEASTEMP	Measured temperature

double *temp

Pointer for the temperature data storage location variable.

Description

Acquires the temperature control information specified by *kind* from the handler connected to the station specified by *stno* and stores to the area shown by the pointer variable *temp*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.10 AT_set_handler_PBTYPPE()

Sets the PBTYPPE data to the handler.

```
int AT_set_handler_PBTYPPE(int stno, unsigned int pbttype);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int pbttype

Specifies the PBTYPPE.

Specify with a variable or constant.

Description

Sets the PBTYPPE data specified by *pbttype* to the handler connected to the station specified by *stno*.

➔ For more information on pbttype format, refer to [3. 3. 26 "AT_read_pbttype\(\)" on page 3-38](#).

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.11 AT_set_handler_TESTCNT()

Sets the number of devices for parallel test to the handler.

```
int AT_set_handler_TESTCNT(int stno, unsigned int count);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

unsigned int count

Specifies the number of devices for parallel test.

Specify with a constant or variable having a value of 1, 2, 4, 8, 16, 32, 64, or 128.

Description

Sets the number of devices for parallel test specified by *count* to the handler connected to the station specified by *stno*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.12 AT_read_handler_retest_category()

Acquires the category at the time of retest mode from the handler.

```
int AT_read_handler_retest_category(int stno, int retest, unsigned int *cat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int retest

Specifies the retest category to be read out.

Specify using the following mnemonics.

Mnemonic	Category to be acquired
HL_RETEST1_CATEGORY	Re-test category
HL_RETEST2_CATEGORY	Repeated re-test category

unsigned int *cat

Pointer for the retest category storage location variable.

Explanation

Acquires the retest category data specified by *retest* from the handler connected to the station specified by *stno* and stores it to the area shown by the pointer variable *cat*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the `AT_set_GPIB_address` function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.13 AT_read_handler_DOUBLETEST_category()

Acquires the category at the time of retest mode from the handler.

```
int AT_read_handler_DOUBLETEST_category(int stno, int n, unsigned int *cat);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements of the DOUBLETEST category.

Specify with a variable or constant.

unsigned int *cat

Pointer for the DOUBLETEST category storage location variable.

Explanation

Acquires the DOUBLETEST category data from the handler connected to the station specified by *stno* and stores it for *n* pieces to the area shown by the pointer variable *cat*. If there is no setting, -1 will be stored.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the `AT_set_GPIB_address` function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.14 AT_set_handler_TESTALARM()

Interrupts the handler test.

```
int AT_set_handler_TESTALARM(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Transmits an alarm to the handler connected to the station specified by *stno*, and interrupts the test.

Proper execution results in a normal end (Error Code = 0). However, any exception or abnormality returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.15 AT_read_handler_testdut()

Acquires the device set status of the handler.

```
int AT_read_handler_testdut(int stno, int n, unsigned int *dut);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements.

Specify a value of 1 to 8 by using a variable or constant.

unsigned int *dut

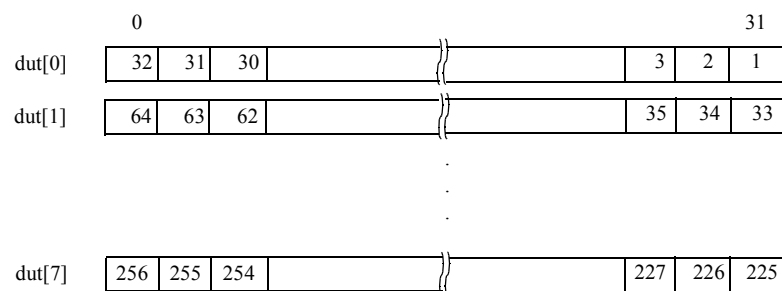
Pointer for the device set status storage location variable.

Description

Acquires the device set status of the handler connected to the station specified by *stno* and stores it for *n* pieces to the area shown by the pointer variable *dut*.

If device set status information is smaller than the number of array elements *n*, 0 will be stored in the remaining areas.

The relationship between DUTs and data is as follows:



Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.16 AT_set_handler_sort()

Sets the sort data to the handler.

```
int AT_set_handler_sort(int stno, int n, unsigned int *sort);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

int n

Specifies the number of array elements.

Specify a value of 1 to 256 by using a variable or constant.

unsigned int *sort

Pointer for the sort data area.

Description

Sets the sort data of the area shown by the pointer variable *sort* to the handler connected to the station specified by *stno*.

If the handler cannot receive the data normally, retransfer will be conducted only once.

The relationship between the read out information and each DUT is as shown below.

```
sort[0] ..... DUT1 sort data
sort[1] ..... DUT2 sort data
      .
      .
sort[254] ..... DUT255 sort data
sort[255] ..... DUT256 sort data
```

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

4. 4.17 AT_clear_handler_retest_sort()

Clears the retest sort data of the handler.

```
int AT_clear_handler_retest_sort(int stno);
```

Argument: int stno

Specifies the station number.

Specify 1 or 2 by using a variable or constant.

Description

Clears the test sort data of the handler connected to the station specified by *stno*.

Proper read out results in a normal end (Error Code = 0). However, any exception or abnormality stops the handler and returns an error code.

➔ For more information on error codes, refer to [5. "Error Code Quick Reference" on page 5-1](#).

! Important

Before executing this function, the AT_set_GPIB_address function (refer to [4. 2. 1 "AT_set_GPIB_address\(\)" on page 4-3](#)) must have been executed.

5. Error Code Quick Reference

Return values from the communication control library are described below.

Error Code	Error Meaning
0	Normal end
1	The specified station number is incorrect.
2	The specified address data is incorrect.
3	An error occurred in the tester controller during program execution.
4	Shared memory cannot be accessed. The tester controller might not have launched, or a processor other than TP5 or TP6 may be connected.
5	A program is executing in the tester controller.
6	The application is locked.
7	The specified file name is incorrect.
8	A subroutine program is executing in the tester controller.
9	The specified sequence number is incorrect.
10	The specified test number is incorrect.
11	The specified number of elements is incorrect.
12	Measurement has not been performed.
13	The application cannot be locked or unlocked. The ASX/U-51 might not be properly installed, or there may be an abnormality in the workstation.
14	A memory map error occurred. There is an abnormality in the workstation.
15	The specified start request is incorrect.
16	The specified GPIB address is incorrect.
17	A GPIB timeout occurred.
18	A GPIB error occurred.
19	No GPIB address has been set.
20	No prober has been selected.
21	The specified DUTNUMSEL is incorrect.
22	Data transmission and receipt failed.
23	The specified number of measurement devices is incorrect.

Error Code	Error Meaning
24	The BINON command has been executed before the execution of the FULLSITES? command.
25	DSA1 does not match DSA2.
26	GPIB interface cannot be accessed. No GPIB driver is installed or the environment setting is incorrect.
27	This command cannot be used.
28	The counter area has not been set.
29	The specified category number is incorrect.
30	The specified sort number is incorrect.
31	The specified DUT number is incorrect.

Appendix 1. GPIB Environment Settings

This chapter describes GPIB environment settings for using the libraries.

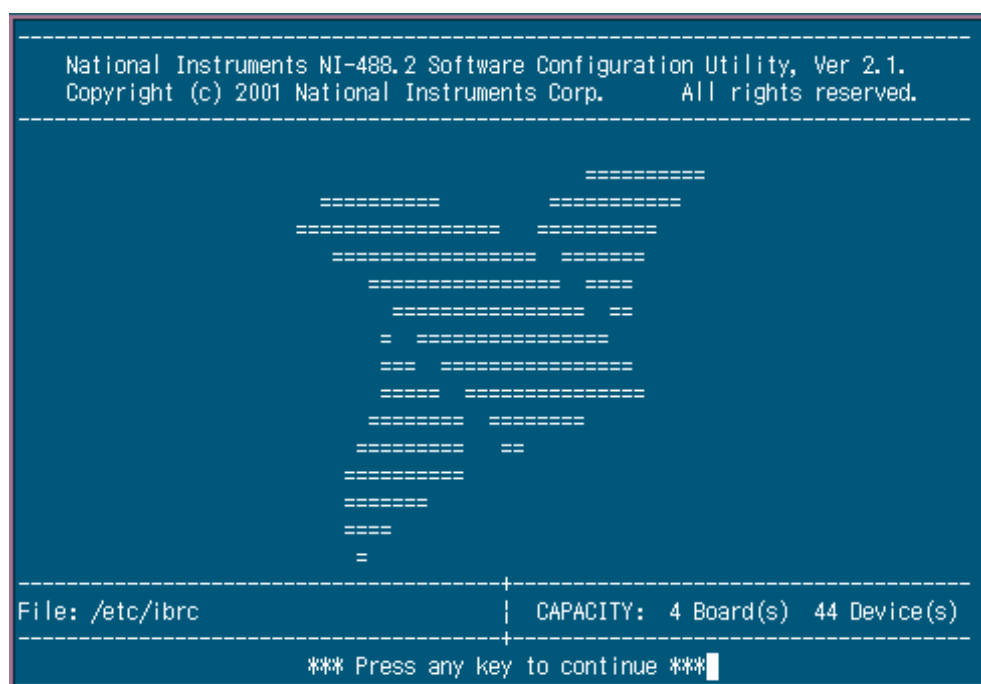
A1. 1 Starting the Configuration Utility

- 1) Login to the root account and enter the following command.

[Figure A 1-1](#) will be displayed.

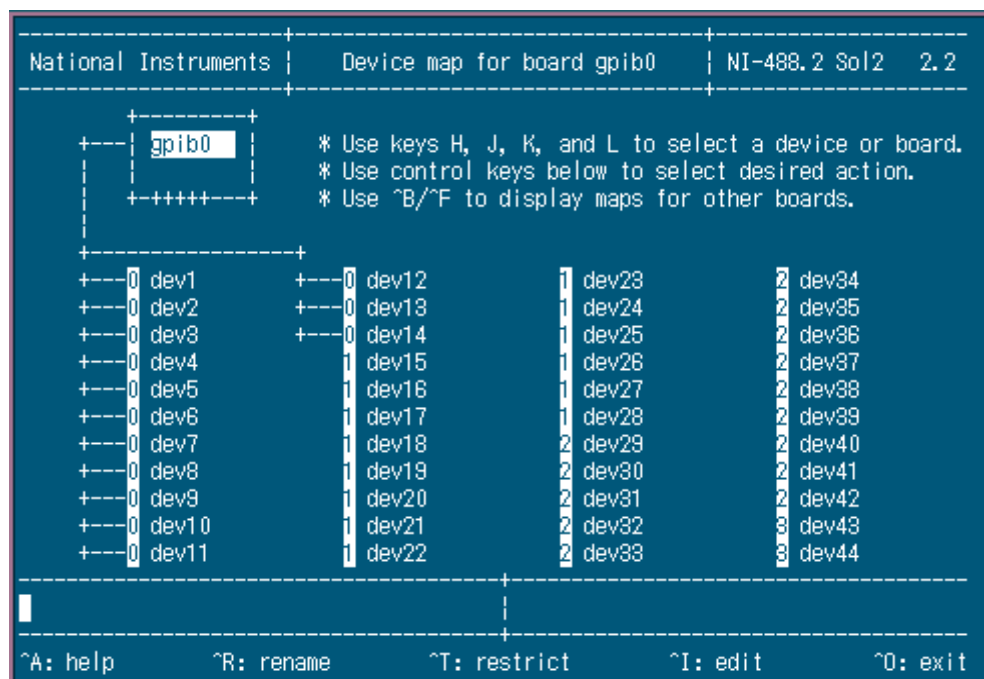
```
# ibconf
```

Figure A 1-1 Start Screen



2) Pressing any key displays [Figure A 1-2](#).

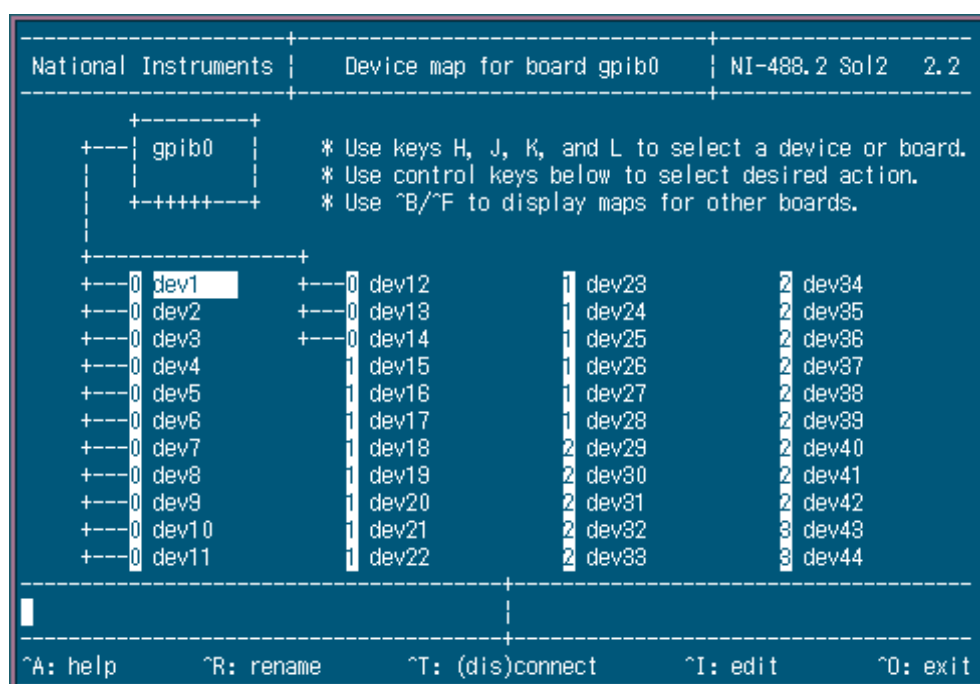
Figure A 1-2 Device Map Screen



A1. 2 Changing the GPIB Device Name

- 1) In [Figure A 1-3](#), using the [H], [J], [K] or [L] key, move the selector to the position of the desired physical device name. The port of the GPIB is allocated to the selected physical device.

Figure A 1-3 Device Map Screen (Physical Device Selection)

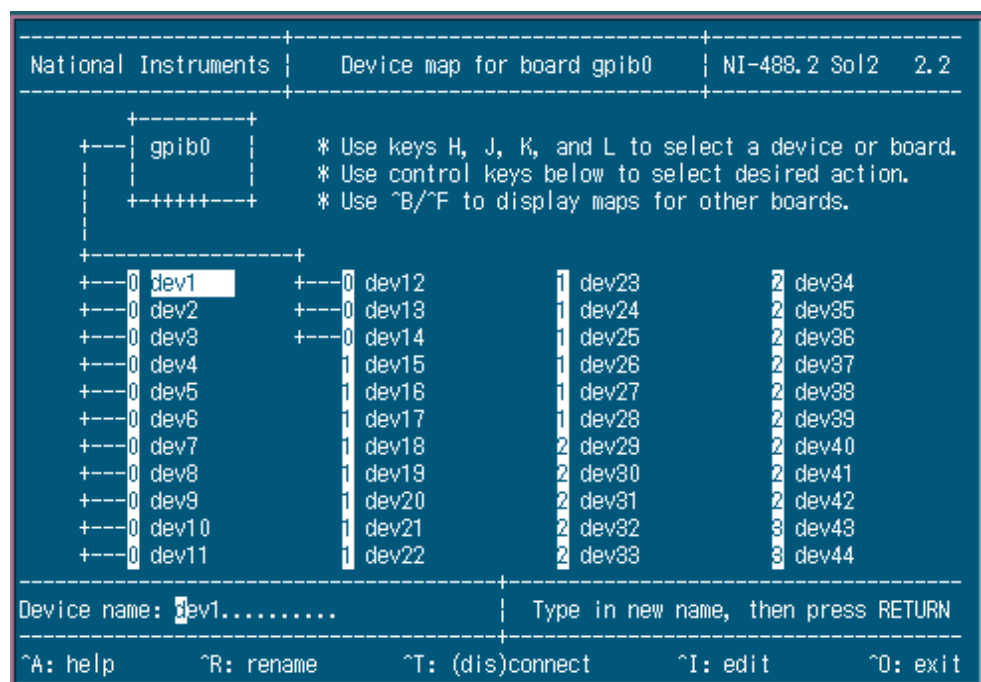


There is no restriction on the physical devices to be used.

Conduct setup on the gpib0 device map screen for using dev1 to dev14, gpib1 device map screen for using dev15 to 28, gpib2 device map screen for using dev29 to 42, and gpib3 device map screen for using dev43 to 44. The device map screen can be switched by entering [Ctrl]+[B] or [Ctrl]+[F].

- 2) As shown in [Figure A 1-4](#), enter [Ctrl]+[R] in the status in which the physical device name is selected. The mode is changed to the device name changing mode.

Figure A 1-4 Device Map Screen (Device Name Change 1)



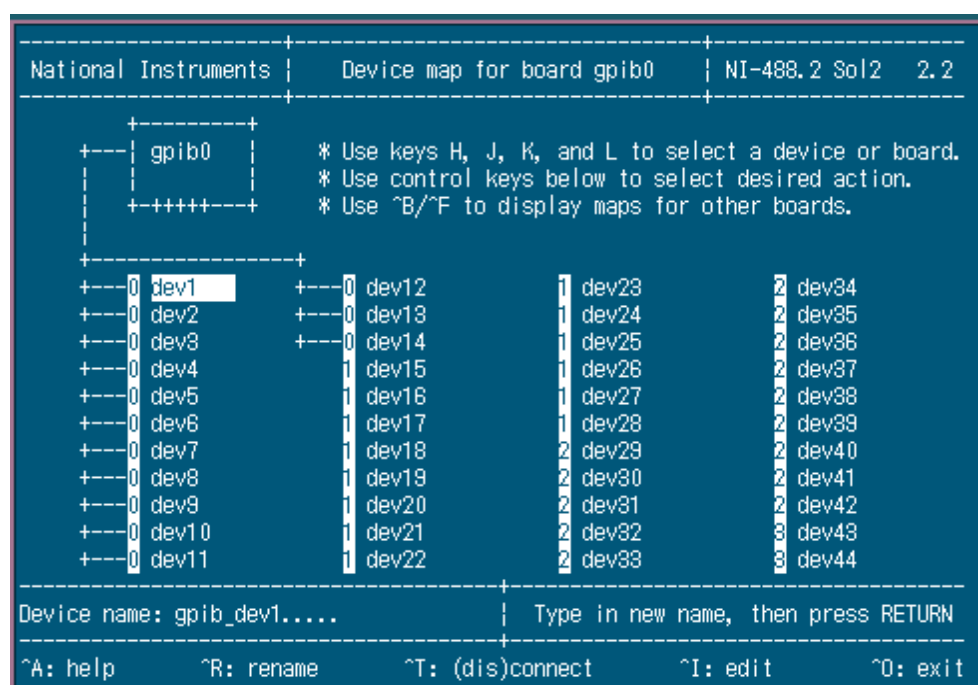
3) As shown in [Figure A 1-5](#), after changing the logical device name, press the [Return] key.

Set the logical device name to gpib_dev1 for station 1 and gpib_dev2 for station 2.

! Important

Because the GPIB control library performs GPIB control using this logical device name, it does not operate normally if the setting is wrong.

Figure A 1-5 Device Map Screen (Device Name Change 2)



A1. 3 Setting GPIB Device Properties

- 1) Move the selector to the position of the logical device name using the [H], [J], [K] or [L] key and enter [Ctrl]+[I]. [Figure A 1-7](#) will be displayed.

Figure A 1-7 Device Properties Setting Screen

National Instruments		Device characteristics		NI-488.2 Sol2 2.2	
Device: gpib_dev1		Access: gpib0		SELECT (use H or L key):	
Primary GPIB address @ 01H				00H to 1EH	
Secondary GPIB address		NONE			
Timeout setting		T10s			
EOS byte		00H			
Terminate read on EOS		no			
Set EOI with EOS on write		no			
Type of compare on EOS		7-bit			
Set EOI w/last byte of write ..		yes			
Repeat addressing		no			
(Use J or K key to change fields)					
^A: help		^W: explain field		^Y: reset value	
				^O: return to map	

- 2) Moving the selector to the position of the primary GPIB address using the [J] or [K] key, change to the address of the connected handler or prober using the [H] or [L] key.

After completing setup, enter [Ctrl]+[O]. The screen returns to the device map screen.

A1. 4 Ending the Configuration Utility

- 1) After completing setup for the stations 1 and 2, enter [Ctrl]+[O] on the device map screen.
- 2) You will be asked whether or not to save setting information as shown in [Figure A 1-8](#). Enter y. Setting information will be saved and the configuration utility will end.

Figure A 1-8 Ending Screen

```

National Instruments | Device map for board gpib0 | NI-488.2 Sol2 2.2
+-----+
+---+ gpib0 | * Use keys H, J, K, and L to select a device or board.
+-----+ | * Use control keys below to select desired action.
          | * Use ^B/^F to display maps for other boards.
+-----+
+---+ 0 gpib_de +---+ 0 dev12 | 1 dev23 | 2 dev34
+---+ 0 gpib_de +---+ 0 dev13 | 1 dev24 | 2 dev35
+---+ 0 dev3     +---+ 0 dev14 | 1 dev25 | 2 dev36
+---+ 0 dev4     | 1 dev15 | 1 dev26 | 2 dev37
+---+ 0 dev5     | 1 dev16 | 1 dev27 | 2 dev38
+---+ 0 dev6     | 1 dev17 | 1 dev28 | 2 dev39
+---+ 0 dev7     | 1 dev18 | 2 dev29 | 2 dev40
+---+ 0 dev8     | 1 dev19 | 2 dev30 | 2 dev41
+---+ 0 dev9     | 1 dev20 | 2 dev31 | 2 dev42
+---+ 0 dev10    | 1 dev21 | 2 dev32 | 3 dev43
+---+ 0 dev11    | 1 dev22 | 2 dev33 | 3 dev44
+-----+
Save current configuration? y | Type 'y' or 'n', then press RETURN
+-----+
^A: help      ^R: rename    ^T: restrict   ^I: edit       ^O: exit

```

List of Figures

Figure 1-1	System Block Diagram	1-2
Figure A 1-1	Start Screen	A1-1
Figure A 1-2	Device Map Screen	A1-2
Figure A 1-3	Device Map Screen (Physical Device Selection)	A1-3
Figure A 1-4	Device Map Screen (Device Name Change 1)	A1-4
Figure A 1-5	Device Map Screen (Device Name Change 2)	A1-5
Figure A 1-6	Device Map Screen (After Device Name has been Changed)	A1-6
Figure A 1-7	Device Properties Setting Screen	A1-7
Figure A 1-8	Ending Screen	A1-8

List of Tables

Table C-1	Conventions for Command-Line and Program Examples	Convention-1
Table C-2	Indicators for Operating the GUI	Convention-2
Table C-3	Indicators for Keyboard Operation	Convention-2
Table C-4	Reference Indicators	Convention-3
Table 3-1	List of HANDLER/PROBER Information Setting Functions	3-1
Table 3-2	List of Initialization Functions	3-1
Table 3-3	List of Program Control Functions	3-2
Table 3-4	List of Measurement Result Processing Functions	3-2
Table 3-5	List of Other Functions	3-4
Table 4-1	List of Functions for Controlling GPIB	4-1
Table 4-2	List of Functions for Controlling the PROBER	4-1
Table 4-3	List of Functions for Controlling the HANDLER	4-2

Index

[A]	
AT_clear_category_counter	3-73
AT_clear_fail_counter	3-72
AT_clear_handler_retest_sort	4-41
AT_clear_program	3-36
AT_clear_sample_number	3-76
AT_clear_slice_number	3-81
AT_clear_sort_counter	3-74
AT_clear_test_fail_counter	3-75
AT_clear_total_counter	3-71
AT_clear_wafer_category_counter	3-79
AT_clear_wafer_counter	3-80
AT_clear_wafer_fail_counter	3-78
AT_clear_wafer_total_counter	3-77
AT_read_category_counter	3-51
AT_read_category_data	3-46
AT_read_COMMENT	3-32
AT_read_DEVICE	3-27
AT_read_DUTIF	3-30
AT_read_fail_counter	3-50
AT_read_fail_dut	3-44
AT_read_GPIB_data	4-7
AT_read_GPIB_status_byte	4-8
AT_read_handler_devicename	4-29
AT_read_handler_DOUBLETEST_	
category	4-37
AT_read_handler_DUTNUMSEL	4-32
AT_read_handler_lotname	4-30
AT_read_handler_retest_category	4-36
AT_read_handler_SRQKIND	4-27
AT_read_handler_status	4-25
AT_read_handler_temp	4-33
AT_read_handler_testdut	4-39
AT_read_LOT	3-28
AT_read_opdat	3-35
AT_read_OPERAT	3-31
AT_read_pbid	3-37
AT_read_pbtype	3-38
AT_read_prober_address	4-14
AT_read_prober_location	4-13
AT_read_prober_lotname	4-11
AT_read_prober_machineID	4-10
AT_read_prober_onwafer	4-15
AT_read_prober_waferID	4-12
AT_read_sample_number	3-54
AT_read_slice_number	3-59
AT_read_sort_counter	3-52
AT_read_sort_data	3-48
AT_read_SUBLOT	3-29
AT_read_test_fail_counter	3-53
AT_read_test_status	3-42
AT_read_total_counter	3-49
AT_read_WAFER	3-33
AT_read_wafer_category_counter	3-57
AT_read_wafer_counter	3-58
AT_read_wafer_fail_counter	3-56
AT_read_wafer_total_counter	3-55
AT_reset_lock	3-84
AT_reset_program	3-40
AT_select_prober	4-9
AT_send_GPIB_data	4-6
AT_send_prober_test_result	4-20
AT_set_category_counter	3-62
AT_set_COMMENT	3-25
AT_set_datalog_mode	3-16
AT_set_DEVICE	3-20
AT_set_DUTIF	3-23
AT_set_fail_counter	3-61
AT_set_fk	3-15
AT_set_GPIB_address	4-3
AT_set_GPIB_timeout	4-4
AT_set_handler_DUTNUMSEL	4-31
AT_set_handler_PBTYPE	4-34
AT_set_handler_request	3-5
AT_set_handler_sort	4-40
AT_set_handler_SRQ_answer	4-28
AT_set_handler_SRQMASK	4-26
AT_set_handler_TESTALARM	4-38
AT_set_handler_TESTCNT	4-35
AT_set_init	3-18

AT_set_lock	3-82
AT_set_LOT	3-21
AT_set_opdat	3-34
AT_set_OPERAT	3-24
AT_set_prober_address	3-9
AT_set_prober_location	3-10
AT_set_prober_next_die	4-22
AT_set_prober_request	3-7
AT_set_prober_target_die	4-23
AT_set_prober_wafer_unload	4-24
AT_set_prober_Zdown	4-18
AT_set_prober_Zup	4-17
AT_set_praname	3-11
AT_set_sample_number	3-65
AT_set_slice_number	3-70
AT_set_sort_counter	3-63
AT_set_SUBLOT	3-22
AT_set_test_dut	3-19
AT_set_test_fail_counter	3-64
AT_set_test_mode	3-13
AT_set_test_number	3-17
AT_set_total_counter	3-60
AT_set_WAFER	3-26
AT_set_wafer_category_counter	3-68
AT_set_wafer_counter	3-69
AT_set_wafer_fail_counter	3-67
AT_set_wafer_total_counter	3-66
AT_start_prober_test	4-19
AT_start_program	3-39
AT_start_subprogram	3-43
AT_wait_program	3-41

[R]

RDOPDAT	3-86
---------------	------

[W]

WROPDAT	3-85
---------------	------